
FORGE: Reaction-Guided Generative Design of Ionizable Lipids

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Ionizable lipids are essential for RNA delivery, but their discovery largely
2 depends on synthesizing and screening libraries assembled from predefined
3 reactants. This strategy preserves a clear synthesis route while limiting the
4 chemical space that can be explored. *De novo* molecular generation could
5 expand this space, yet existing whole-molecule models do not retain the re-
6 action information needed to assemble their designs. We introduce FORGE
7 (**F**low-matched, **O**pen-ended, **R**oute-resolved **G**eneration and **E**xploration),
8 a generative model that uses known reaction chemistry to design ioniz-
9 able lipids without restricting generation to a fixed reactant catalogue.
10 FORGE represents each synthesis route through precursor roles, reaction-
11 core positions, transformation order, and coarse molecular architecture,
12 while generating the complete molecule at atom-and-bond resolution. The
13 model does not receive component identifiers, stored precursor graphs, or
14 fragment tokens. We evaluated FORGE on Ugi, repeated aza-Michael addi-
15 tion, and repeated reductive amination. Across three independent seeds,
16 FORGE achieved exact level-one (exact-L1) transform-consistent yields
17 of $78.7 \pm 10.1\%$, $77.5 \pm 3.7\%$, and $31.3 \pm 14.3\%$, respectively. Matched
18 whole-molecule generation followed by *post hoc* reaction filtering yielded
19 $11.4 \pm 1.1\%$, $4.0 \pm 0.8\%$, and $13.2 \pm 1.3\%$. FORGE also generated exact-L1
20 products with precursor identities absent from the corresponding training
21 component catalogues across all three reactions, a capability unavailable
22 to finite-catalogue assemblers. These results establish that known reaction
23 chemistry can guide *de novo* molecular generation while preserving the
24 information needed to verify assembly.

1 Introduction

26 Lipid nanoparticles (LNPs) deliver nucleic-acid therapeutics including small interfering RNA,
27 mRNA vaccines, and *in vivo* genome-editing reagents (????). Within each LNP, the ionizable
28 lipid supports nucleic-acid complexation, particle assembly, and endosomal escape, making its
29 chemical structure a major determinant of delivery (?). Small changes to the headgroup or
30 hydrophobic domains can substantially alter function. Their effects depend on the complete
31 molecular structure and do not follow simple design rules (???). Researchers therefore
32 discover ionizable lipids by defining modular precursor libraries, synthesizing combinations,
33 and screening the resulting LNPs experimentally (????). This strategy preserves synthetic
34 tractability and supports systematic testing within defined chemical families, while limiting
35 the search to components selected before design begins.

36 *De novo* molecular generation has the potential to extend ionizable-lipid discovery beyond
37 fixed precursor libraries. Current synthesis-aware generators construct molecules by selecting

reactants and applying predefined transformations (????). This construction retains an explicit assembly route and limits generated molecules to the supplied reactant inventory. While whole-molecule generators can propose structures beyond a fixed inventory, their outputs do not retain the precursor roles and transformations needed to assemble each design (??). Other approaches estimate synthesizability after generation without incorporating the intended assembly chemistry into the generative process (?). Recent studies have also applied *de novo* generative models to ionizable lipids (??). The missing capability, however, is generation of complete molecular structures under known reaction chemistry while leaving precursor identities open.

To mitigate this, we introduce FORGE (**F**low-matched, **O**pen-ended, **R**oute-resolved **G**eneration and **E**xploration), a generative model that uses known reaction chemistry to design ionizable lipids beyond a fixed reactant catalogue. Each reaction program contains the assembly chemistry, precursor roles, reaction-core positions, transformation order, and coarse molecular architecture while leaving precursor identities unspecified. FORGE generates complete molecular graphs at atom-and-bond resolution without component identifiers, stored precursor graphs, or fragment tokens. We then decompose each completed graph into its implied precursors and require exact forward replay of the specified assembly reaction to reconstruct the generated molecule. For unavailable precursors, a bounded recursive search returns either a computational synthesis dossier supported by reaction and supplier evidence or an explicit unresolved or search-censored outcome.

We evaluate one shared FORGE model across a Ugi three-component reaction, repeated azo-Michael addition, and repeated reductive amination. Across three independent seeds, FORGE achieves exact-L1 yields of 78.7%, 77.5%, and 31.3%, respectively, exceeding matched whole-molecule generation followed by *post hoc* filtering in every program. FORGE also produces exact assemblies with precursor identities outside the corresponding training catalogues. These experiments evaluate generalization across specified reaction programs and do not claim zero-shot support for arbitrary chemistry.

We summarize our main contributions as follows.

- FORGE generates complete molecular graphs using known reaction chemistry while leaving precursor identities open and does not select from stored reactants or fragment vocabularies.
- Matched controls across three assembly programs showed that reaction-program conditioning substantially increased exact transform-consistent generation relative to whole-molecule generation followed by *post hoc* filtering.
- Across all three programs, FORGE generated exact assemblies with recovered precursor identities outside the corresponding training component catalogues, extending molecular design beyond the reach of finite-catalogue assemblers.

2 FORGE

FORGE generates complete ionizable-lipid graphs under specified assembly chemistry while leaving precursor identities open. Let $G \in \mathcal{G}$ denote a complete molecular graph and $P \in \mathcal{P}$ a reaction program. FORGE models

$$G \sim p_{\theta}(\cdot | P) \quad (1)$$

without receiving precursor identifiers, stored component graphs, or fragment tokens. Figure ?? summarizes reaction-program construction, discrete flow generation, exact assembly verification, and optional route assessment.

2.1 Reaction programs

A reaction program is a tuple

$$P = (c, d, \mathbf{r}, \mathbf{k}, M), \quad (2)$$

where c identifies the ordered assembly chemistry, d is the program depth, $\mathbf{r}$ assigns precursor roles, and $\mathbf{k}$ marks reaction-core positions. The morphology variable M specifies the exterior

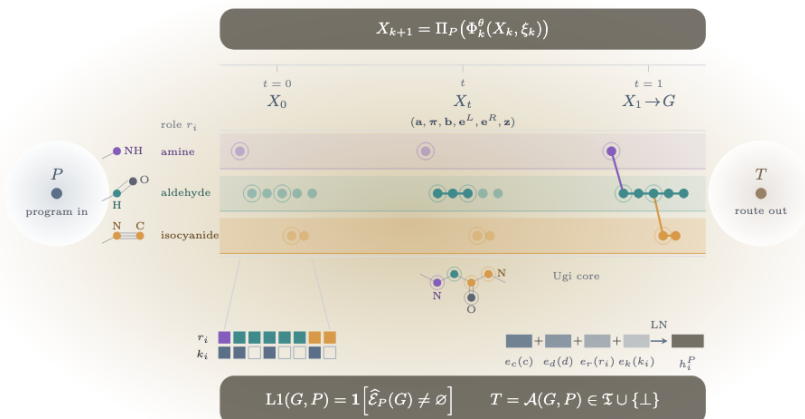


Figure 1: **FORGE generates complete ionizable lipids under specified assembly chemistry.** A reaction program assigns precursor roles and reaction-core positions while leaving precursor identities open. A conditional discrete flow generates the remaining atom, bond, and topology states. Exact forward replay verifies assembly from the recovered precursors, and optional route assessment searches for preparations of unavailable precursors. Colors indicate precursor roles, not stored components.

86 atom count, junction budget, cycle rank, and attachment count for each precursor role.
 87 These values constrain coarse molecular architecture while leaving the exact topology, atoms,
 88 bonds, and precursor identities unspecified.

89 We sample M from a training-fold morphology prior and generate G under the resulting
 90 program,

$$p_{\theta}(G, M | c) = p_{\text{prog}}(M | c) p_{\theta}(G | M, c). \quad (3)$$

91 At graph position i , the program embedding is

$$h_i^P = \text{LN}(e_c(c) + e_d(d) + e_r(r_i) + e_k(k_i)). \quad (4)$$

92 This embedding identifies the position’s precursor role and reaction-core status without
 93 encoding a component identity.

94 2.2 Discrete flow generation

95 FORGE represents a complete molecule using the categorical tree-plus-closure state

$$X_G = (\mathbf{a}, \boldsymbol{\pi}, \mathbf{b}, \mathbf{e}^L, \mathbf{e}^R, \mathbf{z}), \quad (5)$$

96 where $\mathbf{a}$ contains atom states, $\boldsymbol{\pi}$ contains parent pointers, $\mathbf{b}$ contains spanning-tree bond
 97 states, and $(\mathbf{e}^L, \mathbf{e}^R, \mathbf{z})$ encodes closure bonds. The decoder is surjective onto the declared
 98 graph space $\mathfrak{G}_{N_{\text{max}}, K}$, with $N_{\text{max}} = 194$ atoms and at most $K = 3$ closure bonds. Every
 99 graph within this support therefore has a valid representation, although representability does
 100 not imply reaction consistency or nonzero probability under the trained model.

101 Let $\mathcal{X}$ denote the finite categorical state space, p_0 a tractable source distribution, and p_1 the
 102 data distribution. Discrete flow matching constructs a continuous probability path between
 103 p_0 and p_1 while the molecular states remain discrete (?). Given a target state $x_1 \sim p_1$, we
 104 use the linear conditional path

$$p_t(x_t | x_1) = (1 - t)p_0(x_t) + t\delta_{x_1}(x_t), \quad t \in [0, 1]. \quad (6)$$

105 Marginalizing over x_1 gives $p_t(x) = (1 - t)p_0(x) + tp_1(x)$.

106 A continuous-time Markov chain realizes this path through transition rates $R_t(x, y) \geq 0$.
 107 The state probabilities evolve according to

$$\frac{d}{dt}p_t(x) = \sum_{y \neq x} [p_t(y)R_t(y, x) - p_t(x)R_t(x, y)]. \quad (7)$$

108 Given an intermediate state x_t , the model predicts the clean endpoint through $p_\theta(x_1 | x_t, t)$.
 109 If $R_t(x, y | x_1)$ realizes the conditional path for endpoint x_1 , the learned transition rates are

$$R_t^\theta(x, y) = \sum_{x_1 \in \mathcal{X}} p_\theta(x_1 | x, t) R_t(x, y | x_1). \quad (8)$$

110 Sampling begins from $x_0 \sim p_0$ and evolves the categorical state from $t = 0$ to $t = 1$ using R_t^θ .

111 FORGE applies this discrete flow to X_G and conditions endpoint prediction on P . A chemistry
 112 adapter converts the reaction program into masks over atom, pointer, and bond coordinates.
 113 Program-fixed coordinates remain constant, while the flow generates all remaining states. A
 114 graph Transformer predicts the clean atom, bond, pointer, and closure states from (X_t, t, P) .

115 Let $\mathcal{Q}$ denote the six state families, $m_j^{(q)}(P) \in \{0, 1\}$ indicate whether coordinate j in family
 116 q remains variable, and

$$\ell_j^{(q)}(\theta) = -\log p_\theta(X_{G,j}^{(q)} | X_t, t, P) \quad (9)$$

117 denote the corresponding denoising loss. FORGE minimizes

$$\mathcal{L}_{\text{FORGE}}(\theta) = \mathbb{E}_{(G,P) \sim \mu_{\text{bal}}, t \sim \mathcal{U}(0,1), X_t \sim p_t(\cdot | X_G, P)} \left[\sum_{q \in \mathcal{Q}} \lambda_q \frac{\sum_j m_j^{(q)}(P) \ell_j^{(q)}(\theta)}{\max\{1, \sum_j m_j^{(q)}(P)\}} \right], \quad (10)$$

118 where μ_{bal} assigns equal total training mass to each reaction program and λ_q weights state
 119 family q . The denominator normalizes each family by its number of variable coordinates,
 120 with zero contribution when no coordinates remain variable.

121 During corruption and generation, the projection Π_P restores every program-fixed coordinate.
 122 Numerical sampling therefore follows

$$X_{k+1} = \Pi_P(\Phi_k^\theta(X_k, \xi_k)), \quad k = 0, \dots, L-1, \quad (11)$$

123 where Φ_k^θ is the stochastic flow update and ξ_k contains its sampling randomness. This
 124 projection keeps the specified reaction-core states invariant while the flow generates the
 125 remaining molecular structure. The terminal state X_L is decoded into G and must be
 126 connected, valence-valid, and chemically sanitized.

127 2.3 Assembly verification and route assessment

128 After generation, we apply the inverse transform associated with P to recover the implied
 129 precursors. The molecule satisfies the level-one assembly criterion only when the correspond-
 130 ing forward transform reconstructs G exactly. This test verifies the specified final reaction
 131 without requiring the generator to select from a stored precursor catalogue.

132 For precursors that are not directly available, a bounded program constructor $\mathcal{A}$ searches for
 133 preparation routes and checks terminal supplier evidence,

$$T = \mathcal{A}(G, P) \in \mathfrak{T} \cup \{\perp\}, \quad (12)$$

134 where $\mathfrak{T}$ is the space of recursive synthesis programs and $\perp$ denotes abstention. A pair (G, T)
 135 is route-complete only when G passes molecular and exact-assembly verification and every
 136 required precursor traces to an available starting material. The constructor abstains when
 137 the available evidence remains incomplete or the bounded search is exhausted.

138 Generation and route assessment remain separate. The reaction program P conditions
 139 the distribution of G , whereas $\mathcal{A}$ evaluates the completed graph without modifying it.
 140 Appendix ?? provides the complete architecture, sampler, verifier, and route-assessment
 141 procedure. Appendix ?? gives the representation, invariance, sampling-efficiency, and
 142 termination results.

3 Results

We evaluated FORGE on Ugi, repeated aza-Michael addition, and repeated reductive amination using four matched arms: Ugi-only conditioning, shared conditioning, shared training without program information, and shared training with incorrect reaction labels. All arms used the same architecture, optimization budget, and attempt count. We evaluated prespecified checkpoints from three independent seeds. Exact-L1 yield is the fraction of generation attempts that passed molecular validation, program-aware decomposition, and exact forward replay. Appendix ?? provides the full protocol and seed-level results.

3.1 One model learns three assembly programs

We first tested whether one conditional flow could learn all three reactions while preserving Ugi performance. Across 3,072 attempts per program and seed, FORGE achieved exact-L1 yields of $78.7 \pm 10.1\%$ for Ugi, $77.5 \pm 3.7\%$ for repeated aza-Michael addition, and $31.3 \pm 14.3\%$ for repeated reductive amination (Table ??). Exact forward-replay precision reached 100% in every program and seed. Reductive amination showed greater between-seed variation and higher decomposition ambiguity, indicating less consistent learning in this lower-data setting.

Shared conditioning increased Ugi exact-L1 yield from $25.4 \pm 12.0\%$ under Ugi-only training to $78.7 \pm 10.1\%$, a 53.26-point improvement (95% paired-seed interval 41.24 to 61.82). FORGE also produced no fixed-state violations or support overflows across 27,648 held-out attempts. One shared flow therefore learned all three assembly programs without losing the strongest Ugi behavior.

Table 1: **One shared conditional flow learns three ionizable-lipid assembly programs.** Exact-L1 yields are mean $\pm$ sample standard deviation across three seeds. The final column reports decomposition coverage and forward-replay precision.

Program	FORGE conditioned	Shared null	Cyclic program	Decomp./replay (%)
Ugi	78.7 ± 10.1	11.4 ± 1.1	40.2 ± 0.7	90.7/100.0
Aza-Michael	77.5 ± 3.7	4.0 ± 0.8	3.8 ± 0.6	100.0/100.0
Reductive amination	31.3 ± 14.3	13.2 ± 1.3	18.1 ± 0.8	84.3/100.0

3.2 Reaction information changes generation

We next tested whether reaction information changed the generated distribution before verification. Under matched attempt budgets, conditioned generation produced 786.8, 774.6, and 312.7 exact-L1 products per 1,000 attempts for Ugi, aza-Michael addition, and reductive amination. Shared-null generation followed by the same *post hoc* filter produced 114.4, 39.7, and 131.7.

Assigning the wrong reaction identity reduced exact-L1 yield to $40.2 \pm 0.7\%$, $3.8 \pm 0.6\%$, and $18.1 \pm 0.8\%$ across the three programs (Table ??). Removing the program coordinates from a fixed trained network also reduced Ugi exact-L1 yield (Appendix ??). These controls show that the reaction program changes molecular generation before downstream verification.

3.3 Whole-graph generation extends beyond finite catalogues

We next asked whether FORGE could preserve exact assembly while generating new precursor identities. Under the common Ugi benchmark, the finite-inventory methods produced 1000.0 ± 0.0 exact-L1 products per 1,000 attempts but could not generate outside their component vocabularies. FORGE produced 786.8 ± 100.8 exact-L1 products and 751.2 ± 80.2 unique exact-L1 products per 1,000 attempts, all outside the component vocabulary available during generation (Table ??).

Against the fixed training catalogues, component-novel exact-L1 yields reached 729.3, 313.9, and 34.8 products per 1,000 attempts across the three programs. The corresponding finite catalogues had zero component-novel yield by construction (Table ??). Representative Ugi

Table 2: **FORGE combines exact Ugi assembly with precursor identities beyond a finite vocabulary.** Values are mean $\pm$ sample standard deviation per 1,000 attempts across three seeds. Outside vocabulary is defined relative to the components available to each method during generation. $\dagger$ denotes a structural zero for finite-inventory methods. N/E, not estimable.

Method	Valid	Exact L1	Unique L1	Outside vocab.	Held-comp.	Diversity
Reaction-space external baselines						
RGFN	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	N/E
Generic whole-molecule external baselines						
DeFoG unconditional	346.5 $\pm$ 42.3	280.6 $\pm$ 26.6	280.6 $\pm$ 26.6	280.6 $\pm$ 26.6	15.5 $\pm$ 2.4	0.727 $\pm$ 0.014
GenMol/SAFE	627.5 $\pm$ 6.2	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.873 $\pm$ 0.009
Matched controls and formulation baselines						
Learned inventory selector	1000.0 $\pm$ 0.0	1000.0 $\pm$ 0.0	849.5 $\pm$ 13.7	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	0.717 $\pm$ 0.001
Finite catalogue oracle	1000.0 $\pm$ 0.0	1000.0 $\pm$ 0.0	837.6 $\pm$ 8.2	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	0.719 $\pm$ 0.000
Shared-null + post-hoc	194.7 $\pm$ 15.6	114.4 $\pm$ 10.9	108.5 $\pm$ 9.7	108.5 $\pm$ 9.7	0.4 $\pm$ 0.2	0.754 $\pm$ 0.014
FACT-matched	60.8 $\pm$ 20.2	52.0 $\pm$ 15.6	50.8 $\pm$ 14.4	50.8 $\pm$ 14.4	0.0 $\pm$ 0.0	0.667 $\pm$ 0.009
FACT-generous	500.7 $\pm$ 213.8	296.7 $\pm$ 186.0	288.3 $\pm$ 175.3	288.3 $\pm$ 175.3	0.1 $\pm$ 0.2	0.774 $\pm$ 0.007
FORGE Transformer	866.0 $\pm$ 90.2	786.8 $\pm$ 100.8	751.2 $\pm$ 80.2	751.2 $\pm$ 80.2	0.1 $\pm$ 0.2	0.621 $\pm$ 0.011

183 products show exact decompositions into precursor identities outside the visible training
 184 inventory (Figure ??).

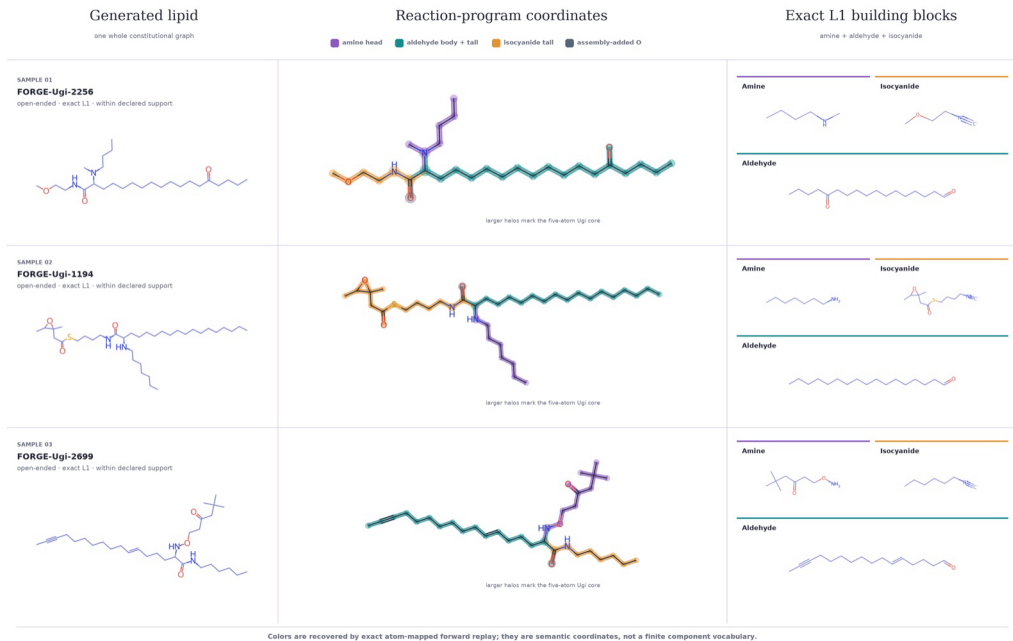


Figure 2: **FORGE generates exact Ugi products beyond its visible component inventory.** Each row contains a seed-0 product with a unique exact-L1 decomposition. Left, the generated molecular graph. Center, precursor origins recovered by exact forward replay. Right, the recovered precursors. Colors indicate precursor roles and reaction-core positions. These examples do not establish activity or experimental synthesis success.

185 Exact-L1 verification establishes the final assembly reaction but does not establish a complete
 186 synthesis route. We record incomplete bounded route searches as abstentions because they
 187 do not demonstrate unsynthesizability.

4 Discussion

In this work, we present FORGE, a generative model that uses known assembly chemistry during whole-molecule generation while leaving precursor identities open. Across Ugi, repeated aza-Michael addition, and repeated reductive amination, reaction-program conditioning increased exact-L1 yield relative to matched whole-molecule generation followed by *post hoc* filtering. FORGE also generated exact assemblies with precursor identities outside the corresponding training catalogues. These results show that reaction information can guide molecular generation without limiting the search to enumerated components.

The evidence was strongest for Ugi and repeated aza-Michael addition. Repeated reductive amination showed lower exact-L1 yield, greater decomposition ambiguity, and substantial between-seed variation. The present study is computational and covers three specified reaction programs within bounded graph support. Exact forward replay verifies structural consistency with the final assembly reaction but does not establish precursor availability, experimental synthesis success, lipid activity, or RNA delivery. The recursive route assessor records computational routes supported by the available reaction and supplier evidence and abstains when that evidence remains incomplete. Appendix ?? reports the additional controls and ablations that define these limits. Experimental synthesis, LNP formulation, and *in vitro* delivery studies will be required to determine whether the generated lipids retain the predicted assembly properties and produce functional RNA delivery.

Table 3: **FORGE combines exact Ugi assembly with precursor identities beyond a finite training vocabulary.** All methods are evaluated under the common component-disjoint split, three-seed attempt budget and exact-L1 verifier, as mean $\pm$ sample standard deviation per 1,000 generation attempts. Outside vocabulary denotes exact-L1 products whose verifier-returned precursor constitution lies outside the component vocabulary available to that method at generation time; it is method-relative and is not comparable across methods with different vocabularies. The distinct component-novel exact-L1 metric, defined against the fixed training component catalogue, is reported in Table ?? . $\dagger$ denotes a structural zero for methods restricted to finite component inventories. N/E, not estimable. No published values from the original baseline papers are inserted. Full admission rules and seed-level results are reported in Appendix ??.

Method	Valid	Exact L1	Unique L1	Outside vocab.	Held-comp.	Diversity
Reaction-space external baselines						
RGFN	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	N/E
Generic whole-molecule external baselines with common post-hoc assessment						
DeFoG unconditional	346.5 $\pm$ 42.3	280.6 $\pm$ 26.6	280.6 $\pm$ 26.6	280.6 $\pm$ 26.6	15.5 $\pm$ 2.4	0.727 $\pm$ 0.014
GenMol/SAFE	627.5 $\pm$ 6.2	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.873 $\pm$ 0.009
Matched controls and formulation baselines						
Learned inventory selector	1000.0 $\pm$ 0.0	1000.0 $\pm$ 0.0	849.5 $\pm$ 13.7	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	0.717 $\pm$ 0.001
Finite catalogue oracle	1000.0 $\pm$ 0.0	1000.0 $\pm$ 0.0	837.6 $\pm$ 8.2	0.0 $\pm$ 0.0 †	0.0 $\pm$ 0.0	0.719 $\pm$ 0.000
Shared-null + post-hoc	194.7 $\pm$ 15.6	114.4 $\pm$ 10.9	108.5 $\pm$ 9.7	108.5 $\pm$ 9.7	0.4 $\pm$ 0.2	0.754 $\pm$ 0.014
FACT-matched	60.8 $\pm$ 20.2	52.0 $\pm$ 15.6	50.8 $\pm$ 14.4	50.8 $\pm$ 14.4	0.0 $\pm$ 0.0	0.667 $\pm$ 0.009
FACT-generous	500.7 $\pm$ 213.8	296.7 $\pm$ 186.0	288.3 $\pm$ 175.3	288.3 $\pm$ 175.3	0.1 $\pm$ 0.2	0.774 $\pm$ 0.007
FORGE Transformer	866.0 $\pm$ 90.2	786.8 $\pm$ 100.8	751.2 $\pm$ 80.2	751.2 $\pm$ 80.2	0.1 $\pm$ 0.2	0.621 $\pm$ 0.011

207 Appendix

208 Appendices A through F are supplementary and are excluded from the five-page short-paper
209 limit, as are the references. They give the full formulation, the complete reporting contract,
210 the theoretical results and a generated-structure atlas.

211 A Preliminaries

212 This appendix restores the background that the short-paper body compresses.

213 A.1 Ionizable-lipid structure and modular assembly

214 The structure of an ionizable lipid is typically organized into three functional regions: a
215 protonatable headgroup, a linker, and one or more hydrophobic domains (??). Together,
216 these regions determine the ionization, stability, lipid packing, and membrane interactions of
217 the molecule. Ionizable-lipid libraries are often synthesized by applying a common reaction
218 to different combinations of precursor molecules (????). Each precursor contributes a defined
219 portion of the final lipid and contains the reactive group required for the assembly. The
220 reaction forms the bonds that connect these precursor-derived regions. As a result, the
221 assembly reaction determines how the regions are connected, while the precursor structures
222 determine the molecular identity of the resulting lipid.

223 A.2 Assembly chemistries studied

224 We study three ionizable-lipid assembly chemistries: an amine–aldehyde–isocyanide Ugi
225 three-component reaction, repeated aza-Michael addition, and repeated reductive amination.
226 In the Ugi reaction, an amine head reacts with an aldehyde component and an isocyanide
227 component to form an α -amino amide that connects the headgroup to two hydrophobic
228 regions (?). A second assembly strategy uses repeated aza-Michael additions to attach
229 acrylate-bearing hydrophobic components sequentially to reactive amines in the headgroup
230 (?). Reductive amination sequentially couples aldehyde-bearing hydrophobic components to
231 reactive amines in the headgroup through new carbon–nitrogen bonds (?).

232 A.3 Reaction transforms and exact assembly

233 Each assembly chemistry defines a forward transformation that maps role-assigned precursor
234 molecules to the product graphs permitted by that reaction. The corresponding reverse
235 transformation decomposes a complete lipid into the precursor assignments that could have
236 produced it. To verify a decomposition, we computationally replay the reaction on the
237 recovered precursors. The product passes the level-one (L1) assembly check only if the
238 reconstruction matches the original lipid exactly.

239 A.4 Discrete molecular graph spaces

240 Let $\mathcal{A}$ denote a finite vocabulary of atom states and $\mathcal{B}$ a finite vocabulary of bond states. A
241 molecular graph $G = (V, E)$ assigns an atom state $a_i \in \mathcal{A}$ to each node $i \in V$ and a bond
242 state $b_{ij} \in \mathcal{B}$ to each edge $(i, j) \in E$. For a fixed maximum heavy-atom count $N_{\max}$, let $\mathcal{G}$
243 denote the resulting finite space of labeled molecular graphs.

244 A.5 Discrete flow matching

245 Let $\mathcal{X}$ be a finite state space, p_0 a tractable source distribution, and p_1 the target data
246 distribution. Discrete flow matching constructs a time-indexed probability path $(p_t)_{t \in [0,1]}$
247 between these distributions, such that $p_{t=0} = p_0$ and $p_{t=1} = p_1$ (?). Given a target sample
248 $x_1 \sim p_1$, the linear conditional path is

$$p_t(x_t | x_1) = (1 - t)p_0(x_t) + t \delta_{x_1}(x_t), \quad t \in [0, 1], \quad (13)$$

249 where δ_{x_1} denotes a point mass at x_1 . Marginalizing over x_1 gives

$$p_t(x) = (1 - t)p_0(x) + tp_1(x), \quad (14)$$

so the states remain discrete while their probabilities evolve continuously in time. A continuous-time Markov chain realizes this probability path through transition rates $R_t(x, y) \geq 0$, where $R_t(x, y)$ is the instantaneous rate of moving from state x to state y . The resulting distribution evolves according to

$$\frac{d}{dt}p_t(x) = \sum_{y \neq x} [p_t(y)R_t(y, x) - p_t(x)R_t(x, y)], \quad (15)$$

which balances the probability flowing into and out of each state. Given an intermediate state x_t , a neural network $p_\theta(x_1 | x_t, t)$ predicts the target state x_1 . We train this predictor by minimizing the denoising cross-entropy

$$\mathcal{L}_{\text{DFM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1), x_1 \sim p_1, x_t \sim p_t(\cdot | x_1)} [-\log p_\theta(x_1 | x_t, t)]. \quad (16)$$

For each endpoint x_1 , let $R_t(x, y | x_1)$ realize the conditional path above. We obtain the learned transition rate by averaging these conditional rates under the model prediction:

$$R_t^\theta(x, y) = \sum_{x_1 \in \mathcal{X}} p_\theta(x_1 | x, t) R_t(x, y | x_1). \quad (17)$$

Sampling draws $x_0 \sim p_0$ and evolves the state from $t = 0$ to $t = 1$ according to the learned transition rates R_t^θ .

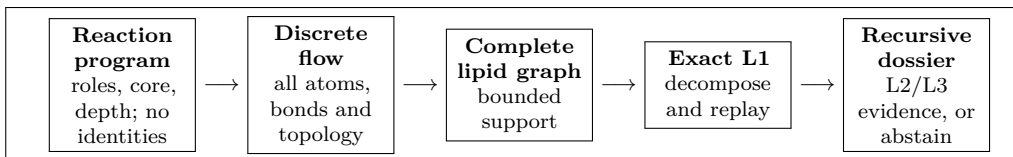


Figure 3: **FORGE generates complete ionizable-lipid graphs within reaction-program coordinates.** The reaction program specifies how molecular regions participate in an assembly without choosing their structures from a finite inventory. FORGE generates the complete molecular graph, verifies that the specified reaction reconstructs it, and recursively expands unavailable precursors until every branch terminates in evidence-qualified starting materials or the framework abstains. Exact replay and route closure are computational checks and do not establish experimental synthesis success.

B FORGE in full

This appendix gives the complete formulation summarized in Section 2 of the body.

B.1 Overview

We formulate ionizable-lipid design as conditional generation under a specified reaction program, while leaving precursor identities unspecified. Let $G \in \mathcal{G}$ denote a complete ionizable-lipid graph and $P \in \mathcal{P}$ a reaction program specifying the assembly chemistry, precursor roles, reaction-core positions, and transformation order. FORGE models the conditional distribution $p_\theta(G | P)$ without access to precursor identifiers, stored component graphs, or fragment tokens. FORGE is organized into four stages:

- 1. Reaction-program specification.** P defines the assembly chemistry, precursor roles, reaction-core positions, transformation order, and coarse per-role morphology, but not precursor identities.
- 2. Whole-graph generation.** A conditional discrete flow generates every atom, bond, and topology state of G under P .
- 3. Exact assembly verification.** A reverse transform decomposes G into role-assigned precursors, and the corresponding forward transform must reconstruct G exactly.
- 4. Recursive synthesis-program construction.** FORGE searches for preparation routes to generated precursors that are not already supported as available starting materials and abstains when a route cannot be completed.

Figure ?? summarizes how FORGE separates reaction semantics from molecular identity and connects whole-graph generation to recursive synthesis-program construction.

B.2 Synthesis-grounded design task

Given a reaction program P , FORGE generates a complete molecular graph G . A downstream bounded program constructor then assesses the exact-assembly precursors and returns a recursive synthesis program T or an explicit abstention outcome. Let $\mathfrak{T}$ denote the space of recursive synthesis programs and $\perp$ an explicit abstention outcome. We write the design process as

$$G \sim p_\theta(\cdot | P), \quad T = \mathcal{A}(G, P) \in \mathfrak{T} \cup \{\perp\}. \quad (18)$$

$\mathcal{A}$ denotes the bounded program constructor that verifies the final assembly, searches for preparation routes to unavailable precursors, and checks terminal supplier evidence. P and $\mathcal{A}$ act at different stages: P guides the generation of G , whereas $\mathcal{A}$ assesses the completed graph without modifying it. A pair (G, T) is designated route-complete only when G passes molecular and exact-assembly verification and every required precursor traces to an available starting material; otherwise the route assessor abstains.

294 B.3 Reaction-program specification

Definition B.1 (Reaction program). *A reaction program is a tuple*

$$295 \quad P = (c, d, \mathbf{r}, \mathbf{k}, M), \quad (19)$$

where c identifies the ordered assembly chemistry, d is the program depth, $\mathbf{r}$ assigns precursor roles, $\mathbf{k}$ marks reaction-core positions, and M specifies coarse per-role morphology. For each precursor role r , M records four integers: the number of exterior atoms n_r , junction budget j_r , cycle rank γ_r , and attachment count a_r .

296
297 M sets only coarse structural constraints; the model generates the exact topology, atoms,
298 and bonds of each region. FORGE first samples M from a training-fold morphology prior
299 and then generates G conditioned on the resulting reaction program:

$$p_\theta(G, M \mid c) = p_{\text{prog}}(M \mid c) p_\theta(G \mid M, c). \quad (20)$$

300 The prior sets each region’s atom count, branching budget, cycle rank, and number of
301 core attachments, but not its molecular structure. A chemistry-specific adapter defines the
302 precursor roles, reaction core, reverse decomposition, and forward transform associated with
303 c . For each graph position i , FORGE combines learned embeddings of the program identity,
304 program depth, precursor role, and reaction-core position:

$$h_i^P = \text{LN}(e_c(c) + e_d(d) + e_r(r_i) + e_k(k_i)). \quad (21)$$

305 These coordinates tell the generator which precursor role each position plays and whether it
306 lies in the reaction core, but not which molecular structure should occupy that role.

307 B.4 Program-defined molecular space

308 Reaction programs constrain how a lipid is assembled without specifying the molecular
309 structures that occupy its precursor roles. Let $\mathcal{G}_{\text{bounded}} \subseteq \mathcal{G}$ denote the molecular graphs
310 within the declared atom and bond vocabularies, the 194-heavy-atom limit, and the three-
311 closure limit. We write $G \models M(P)$ when its per-role atom counts, cycle ranks, and
312 core-attachment counts match M , and its junction counts do not exceed the corresponding
313 budgets. The valid, exact-L1 molecular space associated with P is

$$\mathcal{G}_{\text{L1}}(P) = \{G \in \mathcal{G}_{\text{bounded}} : \text{Valid}(G) = 1, G \models M(P), \text{L1}(G, P) = 1\}. \quad (22)$$

314 A generated lipid can pass the exact L1 check even when one or more recovered precursors
315 never appeared in the training component set, provided the specified reaction reconstructs
316 the lipid from them. For a reaction program P , let $\mathfrak{T}_P$ denote its admissible program inputs,
317 including precursor identities, role assignments, and program depth. The registered forward
318 operator maps each input to a finite set of canonical products:

$$\mathcal{F}_P : \mathfrak{T}_P \longrightarrow 2^{\mathfrak{G}_{N,K}}. \quad (23)$$

319 Given role-specific catalogues $\mathcal{C}$, let $\mathfrak{T}_P(\mathcal{C}) \subseteq \mathfrak{T}_P$ contain the admissible program inputs
320 whose components all belong to their corresponding catalogues. The catalogue-reachable
321 product set is

$$\mathcal{G}_{\text{cat}}(P, \mathcal{C}) = \bigcup_{\mathbf{C} \in \mathfrak{T}_P(\mathcal{C})} \mathcal{F}_P(\mathbf{C}). \quad (24)$$

322 For any graph G , define its complete program-relative preimage as

$$\mathcal{E}_P^*(G) = \{\mathbf{C} \in \mathfrak{T}_P : G \in \mathcal{F}_P(\mathbf{C})\}. \quad (25)$$

Definition B.2 (Strong catalogue escape).

$$\text{Escape}^*(G; P, \mathcal{C}) = \mathbb{I}[\mathcal{E}_P^*(G) \neq \emptyset \wedge \mathcal{E}_P^*(G) \cap \mathfrak{T}_P(\mathcal{C}) = \emptyset]. \quad (26)$$

323
324 Theorem ?? bounds the catalogue-reachable product set while establishing a zero component-
325 novelty ceiling for the catalogue assembler. Strong catalogue escape additionally requires

that no admissible all-catalogue input produces G . The verifier-returned set $\widehat{\mathcal{E}}_P(G)$ defined below is a subset of $\mathcal{E}_P^*(G)$ when every returned trace passes exact forward replay. A novel component in a returned trace establishes strong catalogue escape only when reverse enumeration is exhaustive or direct catalogue reachability is checked independently. FORGE instead parameterizes $p_\theta(G \mid P)$ over whole graphs in $\mathcal{G}_{\text{bounded}}$, without including any component catalogue $\mathcal{C}_r$ in the generative state. Reaction programs constrain assembly semantics, while the flow determines precursor identity at atom and bond resolution.

B.5 Whole-graph generation

FORGE realizes this whole-graph distribution with a discrete flow over atom, bond, and topology states. To preserve the full 194-atom support without an $O(N_{\text{max}}^2)$ edge state, FORGE encodes each graph as a spanning tree plus at most three residual closure edges. Let

$$X_G = (\mathbf{a}, \boldsymbol{\pi}, \mathbf{b}, \mathbf{e}^L, \mathbf{e}^R, \mathbf{z}) \quad (27)$$

denote the categorical graph state, where $\mathbf{a}$ contains atom states, $\boldsymbol{\pi}$ parent pointers, $\mathbf{b}$ spanning-tree bond states, $\mathbf{e}^L$ and $\mathbf{e}^R$ closure-endpoint pointers, and $\mathbf{z}$ closure-bond states. The sparse decoder is surjective onto the declared graph space $\mathfrak{G}_{N_{\text{max}}, K}$: every graph in this space has at least one exact tree-closure encoding (Theorem ??). For this study, $N_{\text{max}} = 194$ and $K = 3$.

The chemistry adapter marks any atom, pointer, or bond states fixed by P . FORGE holds these states constant while corrupting the remaining states along the linear probability path and predicting their clean values from (X_t, t, P) . Let $\mathcal{Q} = \{\mathbf{a}, \boldsymbol{\pi}, \mathbf{b}, \mathbf{e}^L, \mathbf{e}^R, \mathbf{z}\}$ denote the six state families optimized during training. Let μ_{bal} denote the frozen training distribution over graph-program pairs, which assigns equal total mass to each reaction program and preserves the source-balanced weights within each program. For each $q \in \mathcal{Q}$, let $m_j^{(q)}(P) \in \{0, 1\}$ indicate whether coordinate j remains variable under P . FORGE minimizes a program-balanced denoising objective over the variable coordinates:

$$\mathcal{L}_{\text{FORGE}}(\theta) = \mathbb{E}_{(G, P) \sim \mu_{\text{bal}}, t \sim \mathcal{U}(0, 1), X_t \sim p_t(\cdot \mid X_G, P)} \left[\sum_{q \in \mathcal{Q}} \lambda_q \frac{\sum_j m_j^{(q)}(P) \left[-\log p_\theta \left(X_{G,j}^{(q)} \mid X_t, t, P \right) \right]}{\max \left\{ 1, \sum_j m_j^{(q)}(P) \right\}} \right]. \quad (28)$$

Here, λ_q weights state family q , and a family with no variable coordinates contributes zero loss. The corruption and sampling operators project every intermediate state onto the program-fixed coordinates. This makes those coordinates invariant by construction, while the masked denoising objective learns the conditional clean-state distributions of the remaining coordinates in the population limit (Proposition ?? and Theorem ??).

The denoiser is a graph Transformer over the complete sparse state. Each layer applies graph self-attention with relation biases for parent, child and closure edges, then cross-attends to memory tokens representing program identity, program depth, precursor role and reaction-core position. Cross-attention is repeated in every layer so that reaction semantics can influence both local state updates and long-range coordination among precursor-derived regions. Three softly program-routed residual adapters provide family-specific capacity while retaining a shared whole-graph backbone.

The complete training loss augments the flow objective with precursor-role and reaction-core consistency terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{FORGE}} + \lambda_{\text{role}} \mathcal{L}_{\text{role}} + \lambda_{\text{core}} \mathcal{L}_{\text{core}}. \quad (29)$$

Present semantic states receive equal mass within each auxiliary term. Training gives every reaction family equal total loss mass and uses deterministic projected conflicting gradient updates (PCGrad) to remove pairwise gradient conflicts without amplifying a nearly converged family’s gradient. Table ?? prespecifies separate interventions for layerwise cross-attention, each auxiliary loss, routed adapters and gradient conflict control.

At inference, FORGE samples X_0 from the source distribution, holds the program-fixed coordinates constant, and simulates $R_t^\theta(\cdot \mid P)$ from $t = 0$ to $t = 1$. FORGE decodes the

terminal state X_1 into a molecular graph G and retains it only if it is connected, valence-valid, and passes molecular sanitization.

B.6 Exact assembly verification

To ensure that each retained graph satisfies the specified reaction program, FORGE decomposes it into role-assigned precursors and applies the corresponding forward transform.

Definition B.3 (Exact L1 consistency). *For chemistry c , the reverse transform $\mathcal{D}_c$ maps G to a set of role-assigned precursor tuples. Let $\mathcal{D}_P(G) \subseteq \mathcal{D}_c(G)$ contain only those precursor tuples whose role assignments, reaction-core atoms, and morphology satisfy P . The verifier-returned exact set is*

$$\hat{\mathcal{E}}_P(G) = \{\mathbf{C} \in \mathcal{D}_P(G) : G \in \mathcal{F}_P(\mathbf{C})\}. \quad (30)$$

A graph G satisfies exact L1 consistency under P when this set is nonempty:

$$\text{L1}(G, P) = \mathbb{K}[\hat{\mathcal{E}}_P(G) \neq \emptyset]. \quad (31)$$

Post-hoc filtering changes which completed samples are retained but cannot change the proposal distribution’s exact-L1 acceptance probability. Under independent sampling, this probability determines the complete distribution of accepted counts under a fixed attempt budget and the expected cost of obtaining a fixed number of accepted products (Proposition ??). FORGE reports decomposition coverage separately from forward-replay precision because identifying a candidate disconnection does not guarantee exact reconstruction of G . Decomposition coverage is the fraction of valid graphs for which $\mathcal{D}_c(G) \neq \emptyset$; forward-replay precision is the fraction of returned precursor tuples for which $\mathcal{F}_c(\mathbf{C}) \cong G$. FORGE rejects G when $\mathcal{D}_c(G) = \emptyset$ and marks it as ambiguous when $|\mathcal{D}_c(G)| > 1$.

B.7 Recursive synthesis-program construction

The final assembly requires access to every recovered precursor. Exact L1 verification identifies these precursors, but it does not establish whether they are available or how to prepare them. For laboratory planning, an exact final assembly is useful only when every required precursor is either available or connected to available starting materials through a supported preparation route. Let $T = (V_M, V_R, E)$ denote a bipartite synthesis tree rooted at G , where each product molecule points to its preparation reaction and each reaction points to its required reactants. FORGE constructs these routes recursively and returns an abstention for route completion when any branch remains unresolved; the exact-L1 generated product itself remains valid. A molecule node terminates when a current, dated supplier record lists an exact constitutional match as available; otherwise, FORGE must expand that node through a supported, forward-verified reaction. FORGE uses a bounded level-two (L2) planner to propose each expansion. For each proposal, FORGE records the reaction, reactants, product, and supporting evidence, and accepts the step only when the forward transform reconstructs the target precursor exactly. Each accepted reaction adds its reactants as new molecule nodes, and FORGE applies the same termination-or-expansion rule recursively. For each L2 reaction node r , let $\text{Verify}(r) = 1$ when its forward transform reconstructs its product exactly. For each leaf molecule ℓ , let $\text{Available}(\ell) = 1$ when it satisfies the terminal supplier-evidence rule. Let $V_R^{\text{L2}}(T)$ denote the L2 reaction nodes and $L(T)$ the leaf molecules in T .

$$\text{Complete}(G, T, P) = \text{L1}(G, P) \prod_{r \in V_R^{\text{L2}}(T)} \text{Verify}(r) \prod_{\ell \in L(T)} \text{Available}(\ell). \quad (32)$$

Let $\mathcal{B}$ fix the route-depth and query budgets, registered transforms, evidence-admission policy, and dated supplier snapshot used by the assessor $\mathcal{A}_{\mathcal{B}}$. Under finite search bounds and terminating evidence queries, the assessor terminates and returns a dossier only when $\text{Complete}(G, T, P) = 1$ (Lemma ?? and Theorem ??). An abstention means that FORGE did not complete a dossier under $\mathcal{B}$; it does not imply that no synthesis route exists outside the frozen search space or evidence snapshot. The guarantee is computational and does not establish experimental synthesis success.

Algorithm 1 FORGE generation and fail-closed synthesis-program construction.

Require: trained rates R_t^θ , reaction program P , transition count K , attempt budget B , recorded seed s , and frozen route contract $\mathcal{B}$
Ensure: accepted computational dossiers $\mathcal{S}$ and typed failure ledger $\mathcal{H}$

```
1: initialize the random-number streams from  $s$ ;  $\mathcal{S} \leftarrow \emptyset$ ;  $\mathcal{H} \leftarrow \emptyset$ 
2: for  $b = 1, \dots, B$  do
3:   sample  $X_0 \sim p_0(\cdot | P)$  and set  $X \leftarrow \Pi_P(X_0)$ 
4:   for  $k = 0, \dots, K - 1$  do
5:      $t_k \leftarrow k/K$ 
6:      $X \leftarrow \Pi_P(\text{JUMPSTEP}(X, R_{t_k}^\theta(\cdot | X, P)))$ 
7:   end for
8:    $G \leftarrow \text{DECODE}(X)$ 
9:   if  $G$  is disconnected, valence-invalid, or fails molecular sanitization then
10:     $\mathcal{H} \leftarrow \mathcal{H} \cup \{(b, \text{INVALIDGRAPH})\}$ 
11:    continue
12:   end if
13:    $\hat{\mathcal{E}}_P(G) \leftarrow \{\mathbf{C} \in \mathcal{D}_P(G) : G \in \mathcal{F}_P(\mathbf{C})\}$ 
14:   if  $\hat{\mathcal{E}}_P(G) = \emptyset$  then
15:      $\mathcal{H} \leftarrow \mathcal{H} \cup \{(b, \text{L1FAILURE})\}$ 
16:     continue
17:   end if
18:    $Y \leftarrow \mathcal{A}_B(G, P)$   $\triangleright$  recursively assess the exact L1 precursors
19:   if  $Y$  is a tree  $T$  and  $\text{Complete}(G, T, P) = 1$  then
20:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{DOSSIER}(G, P, T)\}$ 
21:   else
22:      $\mathcal{H} \leftarrow \mathcal{H} \cup \{(b, Y)\}$   $\triangleright$  preserve the typed outcome
23:   end if
24: end for
25: return  $\mathcal{S}, \mathcal{H}$ 
```

412 Algorithm ?? summarizes the complete inference procedure. Every iteration consumes one
413 generation attempt, and FORGE retains the outcome of every failed attempt in a typed
414 ledger.

415 For every accepted pair (G, T) , FORGE records a computational synthesis dossier containing
416 all molecules and reactions in T , the evidence supporting each reaction, and the dated
417 supplier record for every terminal material. We classify each generated lipid into one
418 of four support tiers according to which elements of its assembly lie outside the fixed
419 reference space: **E0 (enumerated)**: the generated lipid matches a product that was
420 already constructed computationally from known components using a supported reaction.
421 **E1 (known components)**: the generated lipid was not included in the bounded E0
422 enumeration, but a supported reaction assembles it entirely from known components. **E2**
423 **(generated precursor)**: a supported reaction assembles the lipid, but at least one precursor
424 is newly generated and requires a complete L2 route to available starting materials. **E3**
425 **(new assembly family)**: the lipid requires a final assembly reaction outside the qualified
426 reaction registry.

C Extended experimental protocol and results

This appendix gives the full reporting contract, including every matched arm, ablation and diagnostic summarized in Section 3 of the body.

C.1 Experimental settings

Our evaluation spans the three assembly programs defined above, with Ugi as the deeply supervised case, repeated aza-Michael addition as a second family, and repeated reductive amination as a lighter stress test. The evaluation tests four capabilities: learning multiple assembly programs within one model, using reaction semantics during generation rather than after it, producing exact assemblies beyond a finite component catalogue, and tracing generated precursors to available starting materials. We compare four matched training arms: Ugi-only conditioning, shared conditioning across all three programs, shared training without program information, and shared training with cyclically incorrect program labels. All arms use the same architecture and optimization budget, and we evaluate the prespecified final checkpoint from each of three independent seeds. Each matched comparison uses the same number of generation attempts per program. Our primary metric is exact-L1 yield per generation attempt: the number of generated graphs that pass molecular validity, program-aware decomposition, and exact forward replay divided by the total number of sampling attempts. Secondary metrics separate graph quality (validity and connectedness), reaction recovery (coverage, precision, abstention, and ambiguity), and exploration (unique-product fraction, whole-product and component novelty, molecular diversity, and effective component count). We report means across seeds, individual seed values, and 95% paired-seed bootstrap intervals; generated molecules are not treated as independent replicates. Training uses only the component-disjoint training folds. Calibration data are diagnostic, and held-out products neither enter training nor select checkpoints. The three shared arms assign equal total sampling mass to each program while preserving the frozen within-program source weights. We assess whether shared training preserves Ugi behavior using prespecified non-inferiority margins and require zero program-fixed-state violations and zero support overflows.

Two additional comparisons isolate the role of reaction semantics. First, we compare conditioned generation with null generation followed by exact post-hoc reaction filtering under the same attempt budget; the cyclic-label arm tests whether chemical meaning matters beyond the presence of a categorical input. Second, we compare FORGE with a train-only finite-component assembler that receives the exact training components, supported forward chemistry, source-weighted role marginals, and the same number of attempts. We report reaction validity and exploration separately because the catalogue can achieve high exact-L1 yield while having zero input-component novelty by construction. These comparisons make no L2 planner, biological-oracle, or candidate-selection calls.

For the Ugi program, we additionally define a common external-baseline protocol with one reaction-space comparator, RGFN, and two generic whole-molecule comparators, unconditional DeFoG and GenMol/SAFE (???). The Synthesis-DAG method of ?, SynFlowNet and SynCoGen remain methodological context rather than empirical rows (??). The released Synthesis-DAG repository does not contain the paper-specific ionizable-lipid implementation, the released SynFlowNet action space cannot represent the qualified three-reactant Ugi transform, and the released SynCoGen repository has no usable code license. We do not replace missing functionality with an imitation, alter Ugi into sequential pseudo-reactions, or execute unlicensed code. An external row enters the numerical comparison only after the method has been executed on the same training split, generation-attempt budget, seed set, and common molecular and exact-L1 evaluation pipeline used for FORGE. Published values obtained on the method’s original dataset are not mixed with results from this common evaluation. Methods that do not support the additional assembly programs are compared on Ugi only rather than assigned artificial aza-Michael or reductive-amination results. The matched internal suite separately includes unconditioned generation plus post-hoc filtering, an oracle finite-catalogue assembler, a learned inventory selector, two learned component-factorized generators, and targeted interventions on the Transformer. Table ?? reports the completed three-seed Transformer mechanism and component-factorization study. Each intervention uses the same folds, generation budget and evaluation code as the full Transformer.

Exact-L1 yield (%) and transform verification across three seeds				
Program	FORGE conditioned	Shared null	Cyclic program	Decomp./replay (%)
Ugi	78.7 $\pm$ 10.1	11.4 $\pm$ 1.1	40.2 $\pm$ 0.7	90.7/100.0
Aza-Michael	77.5 $\pm$ 3.7	4.0 $\pm$ 0.8	3.8 $\pm$ 0.6	100.0/100.0
Reductive amination	31.3 $\pm$ 14.3	13.2 $\pm$ 1.3	18.1 $\pm$ 0.8	84.3/100.0
Shared conditioning improved Ugi exact-L1 yield over Ugi-only training by 53.26 percentage points (95% interval 41.24 to 61.82).				

Figure 4: **A shared model learns multiple ionizable-lipid assembly programs.** Results are reported separately for Ugi, repeated aza-Michael addition and repeated reductive amination. Cells show means and sample standard deviations across three independent training seeds. The stated Ugi contrast uses a 95% paired-seed bootstrap interval. The Ugi-only arm is evaluated only on Ugi, and results are not pooled across reaction programs. All values in this figure and Table ?? are hash verified.

Table 4: **Matched production comparison across reaction programs.** Point estimates report the mean across three independent seeds; the exact-L1 column additionally reports the sample standard deviation. Paired contrasts include 95% paired-seed bootstrap intervals; individual seed values appear in the supplement. Validity and connectedness use all generation attempts as the denominator. Decomposition coverage, replay precision, abstention and ambiguity retain their prespecified conditional denominators. Novelty is conditioned on valid products or unique returned decompositions, as appropriate. Fraction pairs and exact-L1 yield are percentages; diversity is mean pairwise ECFP4 distance and N_{eff} is the effective generated-component count. Results are never pooled across reaction programs.

Program	Arm	Valid./Conn.	Decomp./Replay	Abst./Ambig.	Exact-L1/attempt	Whole/Comp. novelty	Diversity/ N_{eff}
Ugi	Ugi-only	28.0/28.0	89.9/100.0	10.1/0.0	25.4 $\pm$ 12.0	99.8/92.4	0.597/134.8
Ugi	FORGE conditioned	86.6/86.6	90.7/100.0	9.3/0.0	78.7 $\pm$ 10.1	99.8/96.2	0.621/320.8
Ugi	Shared null	19.5/19.5	58.8/100.0	41.2/0.0	11.4 $\pm$ 1.1	100.0/99.9	0.754/219.9
Ugi	Cyclic program	43.4/43.4	92.7/100.0	7.3/0.0	40.2 $\pm$ 0.7	99.9/90.1	0.562/138.0
Aza-Michael	FORGE conditioned	77.5/77.5	100.0/100.0	0.0/54.9	77.5 $\pm$ 3.7	99.7/99.2	0.521/156.9
Aza-Michael	Shared null	16.5/16.5	24.3/100.0	75.7/0.1	4.0 $\pm$ 0.8	99.9/99.7	0.635/111.3
Aza-Michael	Cyclic program	25.7/25.7	15.2/100.0	84.8/0.0	3.8 $\pm$ 0.6	99.8/98.3	0.388/51.5
Reductive amination	FORGE conditioned	36.5/36.5	84.3/100.0	15.7/74.1	31.3 $\pm$ 14.3	98.1/100.0	0.624/102.9
Reductive amination	Shared null	15.4/15.4	85.4/100.0	14.6/70.7	13.2 $\pm$ 1.3	99.5/100.0	0.615/77.2
Reductive amination	Cyclic program	21.4/21.4	84.9/100.0	15.1/74.6	18.1 $\pm$ 0.8	96.2/100.0	0.570/34.8

482 C.2 Shared reaction-program learning across three assembly chemistries

483 Across 3,072 held-out generation attempts per program in each of three independent training
484 seeds, the final conditioned model achieved exact-L1 yields of $78.7 \pm 10.1\%$ for Ugi (72.3%,
485 90.3% and 73.5% by seed), $77.5 \pm 3.7\%$ for repeated aza-Michael addition (76.5%, 81.5% and
486 74.3%), and $31.3 \pm 14.3\%$ for repeated reductive amination (38.6%, 14.8% and 40.4%) (mean
487 $\pm$ sample standard deviation; Figure ?? and Table ??). Exact forward-replay precision was
488 100% in every program and seed. However, mean decomposition ambiguity was 54.9% for
489 repeated aza-Michael addition and 74.1% for repeated reductive amination, and the latter
490 program showed substantial between-seed variation. Table ?? reports molecular validity,
491 connectedness, decomposition coverage, forward-replay precision, abstention, ambiguity,
492 novelty and diversity separately for each program.

493 Relative to the Ugi-only model, shared training changed Ugi exact-L1 yield by 53.26 percent-
494 age points (95% paired-seed interval 41.24 to 61.82) and met every prespecified Ugi-retention
495 gate. The final conditioned model produced zero fixed-state violations and zero support
496 overflows across 27,648 held-out attempts (three seeds, three programs and 3,072 attempts
497 per program). Together, these results support the conclusion that one conditional flow can
498 learn the three specified reaction-program coordinates without erasing the deeply supported
499 Ugi behavior.

500 C.3 Reaction semantics alter generation exact rather than merely filtering its output

501 Under matched attempt budgets, conditioned generation produced 786.8, 774.6 and 312.7
502 exact-L1 products per 1,000 attempts for Ugi, repeated aza-Michael and repeated reductive
503 amination, compared with 114.4, 39.7 and 131.7 for matched null generation followed by
504 post-hoc filtering (Figure ??a-c). The paired conditioned-minus-post-hoc differences were

Table 5: **Method-blind structural-realism diagnostic.** Every entry is mean $\pm$ sample standard deviation across three training seeds and retains all 3,072 requested attempts in the denominator. Support is the declared C/N/O/P/S and 194-heavy-atom space. FP and descriptor P/C report manifold precision per 1,000 attempts and coverage of the source-study-held-out R0 reference in percent. Desc. W is the normalized descriptor Wasserstein distance; lower is closer. C2ST is grouped classifier two-sample area under the receiver operating characteristic curve; 0.5 is indistinguishable. Unique is conditioned on connected outputs. N_{eff} is the Shannon effective molecule count. N/E denotes a metric that cannot be estimated. This post-hoc structural diagnostic is not a property, activity or synthesis-success

Method	Connected/1k	Support/1k	FP P/C	Desc. P/C	Desc. W	C2ST	Unique (%)	N_{eff}	Diversity
RGFN	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	N/E	N/E	N/E	0.0 $\pm$ 0.0	N/E
DeFoG unconditional	346.5 $\pm$ 42.3	334.2 $\pm$ 39.1	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	17.3 $\pm$ 1.4/1.45 $\pm$ 0.06	2.067 $\pm$ 0.009	0.961 $\pm$ 0.007	100.0 $\pm$ 0.0	1064.3 $\pm$ 129.9	0.726 $\pm$ 0.013
GenMol/SAFE	627.5 $\pm$ 6.2	627.4 $\pm$ 6.2	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	3.199 $\pm$ 0.003	N/E	0.9 $\pm$ 0.1	3.2 $\pm$ 0.1	0.873 $\pm$ 0.009
Finite catalogue oracle	1000.0 $\pm$ 0.0	956.5 $\pm$ 2.2	0.2 $\pm$ 0.2/0.05 $\pm$ 0.05	54.6 $\pm$ 4.4/2.02 $\pm$ 0.17	2.107 $\pm$ 0.002	0.971 $\pm$ 0.007	83.8 $\pm$ 0.8	2395.5 $\pm$ 31.7	0.720 $\pm$ 0.000
Learned inventory selector	1000.0 $\pm$ 0.0	945.0 $\pm$ 6.5	0.1 $\pm$ 0.2/0.02 $\pm$ 0.03	56.6 $\pm$ 7.2/1.97 $\pm$ 0.46	2.101 $\pm$ 0.005	0.974 $\pm$ 0.002	85.0 $\pm$ 1.4	2443.0 $\pm$ 55.8	0.718 $\pm$ 0.001
FORGE Transformer	866.0 $\pm$ 90.2	864.6 $\pm$ 91.2	0.0 $\pm$ 0.0/0.00 $\pm$ 0.00	8.8 $\pm$ 2.3/1.16 $\pm$ 0.20	2.089 $\pm$ 0.005	0.973 $\pm$ 0.019	95.7 $\pm$ 1.9	2479.4 $\pm$ 177.2	0.621 $\pm$ 0.013

67.24 percentage points for Ugi (95% interval 60.84 to 79.95), 73.49 for aza-Michael (69.69 to 78.45), and 18.10 for reductive amination (2.51 to 26.04).

The cyclic control retained precursor roles, reaction-core coordinates and program depth but assigned the wrong reaction identity. Relative to correct conditioning, this intervention changed exact-L1 yield by 38.43 percentage points for Ugi (95% interval 32.00 to 49.41), 73.67 for aza-Michael (71.13 to 77.02), and 13.15 for reductive amination (-3.91 to 22.01). Forward-replay precision remained 100% for every returned decomposition in these arms.

A separate inference-time intervention held the trained network and Ugi request fixed while changing only program coordinates. The factual exact-L1 yield was 39.69%. Nulling all program coordinates reduced it to 3.62%, a 36.07-point loss (95% interval 34.28 to 37.24). Reversing the role assignment caused a 4.49-point loss (1.37 to 8.27), and substituting the reductive-amination program token caused a 22.60-point loss (14.06 to 27.47). The aza-Michael-token contrast was unresolved at 2.00 points (-2.18 to 5.60). Thus the coordinates affect generation, but the evidence does not support an equal or individually identifiable effect for every coordinate.

C.4 Whole-graph generation produces precursor identities beyond a finite vocabulary

Table ?? defines the common Ugi benchmark used to compare FORGE with external synthesis-aware generators, matched internal controls, and the finite-catalogue oracle. Every yield is normalized by the number of generation attempts, including attempts that fail before producing a valid molecule. All included methods were run under the common split, attempt budget, seed set and verifier.

The learned finite-inventory selector achieved 1000.0 ± 0.0 exact-L1 products per 1,000 attempts, but its finite action space imposed zero generation outside its own component vocabulary. FORGE produced 786.8 ± 100.8 exact-L1 products and 751.2 ± 80.2 unique exact-L1 products outside its method vocabulary per 1,000 attempts. Among open-ended external generators, unconditional DeFoG produced 280.6 ± 26.6 exact-L1 products per 1,000 attempts and recovered held components at 15.5 ± 2.4 per 1,000 attempts, compared with 0.1 ± 0.2 for FORGE. GenMol/SAFE produced 627.5 ± 6.2 valid connected outputs but no exact Ugi-L1 product, while RGFN emitted no admissible molecular output under the strict native contract. These outcomes expose separate tradeoffs between reaction consistency, open-ended identity generation and recovery of specifically reserved components.

No evaluated method reproduced the full held-out observed-lipid distribution: where estimable, grouped C2ST AUC remained between 0.961 and 0.974, and fingerprint-manifold precision was zero or near zero (Table ??). GenMol/SAFE’s high pairwise diversity was paired with an effective count of only 3.2 ± 0.1 molecules, confirming severe frequency collapse. FORGE retained 2479.4 ± 177.2 effective molecules, but only 8.8 ± 2.3 attempts per 1,000 entered the descriptor manifold. We therefore interpret this analysis as a multi-axis structural diagnostic rather than a single measure of whether a generated lipid is reasonable.

Table 6: **Transformer architecture and mechanism ablations.** The five Transformer intervention rows change only the named mechanism from the full model; the two FACT rows instead generate precursor roles independently at matched or increased capacity. All arms use identical data folds, corruption, context schedule, checkpoint rule and sampling budget. Entries report the mean and [95% seed-bootstrap interval] across three independent training seeds. Held-out loss is the mean denoising loss at $t = 0.5$ across the three component-disjoint program partitions. Cross-role fidelity is the Frobenius distance to the held-out residualized descriptor matrix, for which lower values indicate closer agreement. Exact-L1 yields use all 3,072 attempts per program and seed as the denominator.

Arm	Held-out loss	Ugi L1/1k	BL L1/1k	LX L1/1k	Held-comp. L1/1k	Cross-role fidelity	Diversity
Input-only program	8.48 [7.72, 9.14]	737.30 [631.18, 845.70]	779.84 [757.49, 820.31]	277.34 [230.14, 339.52]	0.11 [0.00, 0.33]	1.79 [1.70, 1.91]	0.60 [0.59, 0.61]
No role loss	7.87 [7.78, 8.00]	771.59 [672.20, 889.00]	772.14 [743.49, 820.64]	289.28 [115.23, 396.16]	0.11 [0.00, 0.33]	1.94 [1.89, 1.98]	0.59 [0.58, 0.60]
No core loss	7.85 [7.78, 7.98]	753.69 [674.15, 906.58]	772.79 [726.89, 819.34]	300.46 [151.69, 409.18]	0.11 [0.00, 0.33]	1.89 [1.75, 1.98]	0.59 [0.58, 0.59]
No routed adapters	7.81 [7.44, 8.04]	718.64 [592.45, 875.33]	779.08 [744.47, 823.89]	326.71 [265.62, 384.11]	0.00 [0.00, 0.00]	1.88 [1.75, 2.06]	0.59 [0.59, 0.59]
No gradient conflict control	7.98 [7.82, 8.20]	798.07 [732.42, 916.02]	768.88 [721.68, 806.32]	326.93 [149.09, 427.73]	0.11 [0.00, 0.33]	1.96 [1.87, 2.09]	0.59 [0.58, 0.59]
FACT-matched	7.09 [6.86, 7.34]	51.97 [37.76, 68.68]	393.66 [327.47, 460.94]	509.87 [457.36, 562.83]	0.00 [0.00, 0.00]	1.45 [1.14, 1.72]	0.56 [0.55, 0.57]
FACT-generous	8.24 [8.03, 8.60]	296.66 [161.78, 508.79]	686.96 [584.64, 811.52]	561.31 [491.54, 635.74]	0.11 [0.00, 0.33]	1.16 [1.14, 1.18]	0.60 [0.60, 0.60]
FORGE Transformer	7.88 [7.68, 8.12]	791.78 [725.91, 916.67]	762.48 [725.26, 815.10]	318.14 [134.44, 413.41]	0.11 [0.00, 0.33]	1.94 [1.76, 2.09]	0.59 [0.58, 0.59]

C.5 Joint whole-lipid denoising has reaction-dependent advantages over role isolation

The full Transformer produced 791.8 Ugi and 762.5 repeated aza-Michael exact-L1 products per 1,000 attempts, compared with 52.0 and 393.7 for the parameter-matched role-isolated FACT model (Table ??). The full Transformer and FACT-matched model each had 5.33 million trainable parameters; FACT-generous had 29.40 million. The higher-capacity FACT control improved to 296.7 for Ugi and 687.0 for repeated aza-Michael addition, but remained below the full Transformer in every seed for both programs. The direction reversed for repeated reductive amination: the full Transformer produced 318.1 exact-L1 products per 1,000 attempts, whereas FACT-matched and FACT-generous produced 509.9 and 561.3. Thus, joint whole-lipid denoising improved exact assembly for Ugi and repeated aza-Michael addition but was not uniformly superior across reaction programs. Because the prespecified conditional-dependence diagnostic was negative, this architecture contrast does not establish detectable cross-role dependence in the present data.

Removing layerwise program cross-attention reduced mean exact-L1 yield by 54.5 products per 1,000 attempts for Ugi and 40.8 for repeated reductive amination, while increasing repeated aza-Michael yield by 17.4. Removing the role or core objective and removing routed adapters also produced program-dependent changes rather than a uniform loss. The arm without gradient-conflict control slightly exceeded the full Transformer mean in all three programs. We therefore do not claim that every introduced module independently improves exact-L1 yield. These one-at-a-time interventions identify which effects replicate across chemistries; they are not pooled into a composite architecture score.

At the matched sampling budget, FORGE produced 751.2 unique exact-L1 Ugi products per 1,000 attempts outside its method vocabulary, averaged across the three independent seeds. The corresponding values were 313.9 for aza-Michael addition and 34.8 for reductive amination. The finite-component assembler produced zero by construction because every sampled product uses only components from the training catalogue (Figure ??d-f and Table ??). The catalogue achieved exact-L1 yields of 1,000.0, 723.2 and 1,000.0 products per 1,000 attempts, compared with 786.8, 774.6 and 312.7 for FORGE. The corresponding raw validities were 1,000.0, 723.2 and 1,000.0 for the catalogue and 866.0, 774.6 and 365.0 for FORGE. We report these quantities separately because reaction validity and verifier-relative component novelty measure different capabilities.

In a distinct-product Ugi role audit, 19,520 of 26,235 products (74.4%) contained at least one verifier-returned component constitution absent from the training registry. Across all Ugi products admitted to the novelty audit, 21,960 of 26,235 distinct products (83.7%) were absent from the full 112,386-product constitutional reference corpus. FORGE effective component counts were 320.8, 156.9 and 102.9, compared with 86.4, 23.4 and 16.9 for the finite catalogue. The Ugi catalogue contained 185 amines, 75 aldehydes and 35 isocyanides and covered 100% of training products but 0% of the calibration and component-disjoint held-out products. Its Cartesian tuple ceilings were 485,625 for Ugi, 774 for aza-Michael and 490 for reductive amination; these count possible component-program inputs rather

than unique forward products. These results establish the intended tradeoff: the exact finite catalogue has high transform consistency, whereas whole-graph generation can produce exact-L1 products whose recovered precursor identities are not restricted to that inventory.

Exact assembly versus the finite-catalogue ceiling				
Program	FORGE L1/1k	Catalogue L1/1k	FORGE comp.-novel/1k	Catalogue comp.-novel/1k
Ugi	786.8	1000.0	729.3	0.0
Aza-Michael	774.6	723.2	313.9	0.0
Reductive amination	312.7	1000.0	34.8	0.0

Nulling all program coordinates reduced factual Ugi exact-L1 yield from 39.69% to 3.62%; substituting the reductive-amination token reduced it to 17.09%.

Figure 5: **Reaction conditioning improves sampling efficiency without imposing a finite component vocabulary.** The inset reports inference-time coordinate interventions, and the table compares FORGE with a train-only finite-component assembler that receives exact forward chemistry and the same per-program attempt budget. Values are means across three independent training seeds. The comparisons use identical molecular support and final reaction adapters and make no L2-planner, biological-oracle or candidate-selection calls. The finite catalogue has zero component-novel yield by construction.

Table 7: **Whole-graph generation versus a finite-component assembler.** Exact-L1, unique exact-L1 and component-novel exact-L1 yields are reported per 1,000 generation attempts. A component-novel exact-L1 product has exactly one verifier-returned exact-L1 decomposition and contains at least one component constitution absent from the fixed training component catalogue. This is the metric that supports claims about precursor identities beyond the training catalogue, and it is distinct from the method-relative outside-method-vocabulary yield of Table ???. This verifier-relative metric is not called strong catalogue escape unless reverse enumeration is shown to be exhaustive. The catalogue has zero input-component novelty by construction, but may achieve higher raw validity or exact-L1 yield. FORGE-minus-catalogue contrasts use the three seeds as paired resampling units.

Program	Method	Valid/1k	Exact L1/1k	Unique L1/1k	Component-novel exact L1/1k	Whole nov.	Comp. nov.	Diversity/ N_{eff}
Ugi	FORGE	866.0	786.8	751.2	729.3	99.8	96.2	0.621/320.8
Ugi	Finite catalogue	1000.0	1000.0	837.6	0.0	16.9	0.0	0.719/86.4
Aza-Michael	FORGE	774.6	774.6	738.0	313.9	99.7	99.2	0.521/156.9
Aza-Michael	Finite catalogue	723.2	723.2	147.5	0.0	39.8	0.0	0.402/23.4
Reductive amination	FORGE	365.0	312.7	279.7	34.8	98.1	100.0	0.624/102.9
Reductive amination	Finite catalogue	1000.0	1000.0	52.1	0.0	44.8	0.0	0.514/16.9

C.6 Native recovery of held components remains weak

The component-disjoint Ugi fold was never used for training, checkpoint selection or inventory construction. Under the common native-generation assessment, the full Transformer recovered only 1 exact-L1 product containing a designated held component in 9,216 attempts, or 0.11 products per 1,000 attempts averaged across three seeds. This is a negative generalization result. It does not conflict with the broader verifier-relative novelty measurements above: generating a component absent from the training registry is easier than recovering one of the specifically reserved held-component identities. We therefore do not claim strong held-component recovery, role-specific held-component coverage, or zero-shot held-reaction-family generalization.

C.7 Recursive synthesis programs connect generated lipids to evidence-qualified materials

FORGE includes a bounded post-generation route assessor for generated precursors that are not already supported as available materials. The assessor first queries the frozen route registry and may invoke single-step or multistep proposal engines where the registry is silent. A proposed transformation is accepted only when the corresponding forward reaction reconstructs the intended precursor under the declared verification contract, and a complete dossier is returned only when every terminal branch closes under the frozen material-evidence policy. Otherwise the assessor abstains. Abstention therefore records

Table 8: **Predicted HeLa messenger-transfection potential under matched generation budgets.** The support-enriched arm uses the already promoted applicability proposal; the nested arm reallocates morphology mass using the frozen weak potency signal. Conservative-high counts are computational predictions, not transfection measurements. The comparison uses the same generator-call and oracle budgets.

Arm	Attempts	Exact L1	Eligible before budget	Oracle budget	Unique conservative-high
Support-enriched	3,072	2955	90	38	27
Nested potency proposal	3,072	2962	82	38	23

insufficient evidence under the declared search and evidence contract, not proof that the molecule is unsynthesizable.

We do not report a separate AiZynthFinder row: a proposal-only route is a hypothesis and cannot change exact evidence closure until every step passes the same forward-verification and terminal-evidence gates (?).

An L2 step is admitted only when its evidence tier permits the claimed reaction scope and its forward transform reconstructs the intended precursor. A terminal branch closes only with a current, dated supplier record for an exact constitutional match. Computational route closure establishes transform consistency and evidence-qualified terminal materials, not experimental synthesis success.

C.8 Guidance diagnostics define the limits of additional optimization

At synthesis-guidance strength 0.25, the matched controller evaluated 64 terminal particles and retained 56 canonical representatives per arm. It changed only 8 terminal identities and returned 2 unique route-ready products, compared with 3 after post-hoc assessment. Only five of 48 four-particle morphology groups contained mixed route-readiness values, leaving most ancestry decisions uniform. This negative result does not support promoting mid-trajectory synthesis tilting. The production method therefore remains complete generation followed by proposal-augmented L1/L2/L3 route assessment.

We evaluate biological guidance only in the bounded HeLa messenger-transfection-potential (mTP) diagnostic. A group-disjoint preterminal classifier detected weak morphology-level signal, with ROC AUC 0.623 (95% interval 0.565 to 0.679) and a 1.177-fold observed top-quartile gain. We then compared a frozen applicability-enriched morphology proposal with a nested potency-enriched proposal using 3,072 generator calls per arm. Table ?? reports the matched terminal outcome.

The nested-minus-support difference was -1.30 conservative-high products per 1,000 generator calls (95% interval -2.28 to 0.98). The promotion gate failed. These computations evaluate predictor behavior within a declared applicability domain; they do not establish improved biological activity, delivery or experimental potency.

635 D Extended computational reporting tables

636 The following tables define the complete reporting contract for the common Ugi benchmark.
 637 Completed matched controls, the learned inventory selector and three external generators
 638 are populated from hash-pinned three-seed results.

Table 9: **Training and sampling parity for the common Ugi benchmark.** Generator calls and attempted products are reported separately from reaction-predictor, verifier, planner, and oracle calls. A method is not described as compute-matched when any required quantity is unavailable. N/R means not recorded by the frozen run; N/P means not applicable.

Method	Params.	Train steps	GPU-hours	Sample steps	Attempts	Reaction calls	Route calls	Oracle calls
RGFN	N/R	5,002 iter.	N/R	N/R	$3 \times 3,072$	0	0	0
DeFoG unconditional	N/R	50,000	N/R	1,000	$3 \times 3,072$	0	0	0
GenMol/SAFE	N/R	50,000	N/R	N/R	$3 \times 3,072$	0	0	0
Learned inventory selector	170,855	2,000	N/R	N/P	$3 \times 3,072$	$3 \times 3,072$	0	0
Finite catalogue oracle	N/P	N/P	N/R	N/P	$3 \times 3,072$	N/R	0	0
Shared-null + post-hoc	N/R	N/R	N/R	N/R	$3 \times 3,072$	N/R	0	0
Input-only program	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
No role loss	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
No core loss	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
No routed adapters	5,026,843	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
No gradient conflict control	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
FACT-matched	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
FACT-generous	29,395,993	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0
FORGE Transformer	5,328,301	1,700	N/R	N/R	$3 \times 3,072$	N/R	0	0

Table 10: **Seed-level common Ugi benchmark results.** Rows are expanded once a method has a hash-verified three-seed result vector. The main table reports the corresponding mean $\pm$ sample standard deviation; generated molecules are not treated as independent training replicates.

Method	Seed(s)	Valid/1k	Exact L1/1k	Unique L1/1k	Outside vocab./1k	Held-comp. L1/1k	Diversity
RGFN	20260825	0.0	0.0	0.0	0.0	0.0	N/E
RGFN	20260826	0.0	0.0	0.0	0.0	0.0	N/E
RGFN	20260827	0.0	0.0	0.0	0.0	0.0	N/E
DeFoG unconditional	20260825	299.2	251.0	251.0	251.0	13.7	0.711
DeFoG unconditional	20260826	359.7	288.4	288.4	288.4	18.2	0.733
DeFoG unconditional	20260827	380.5	302.4	302.4	302.4	14.6	0.736
GenMol/SAFE	20260825	627.3	0.0	0.0	0.0	0.0	0.883
GenMol/SAFE	20260826	633.8	0.0	0.0	0.0	0.0	0.869
GenMol/SAFE	20260827	621.4	0.0	0.0	0.0	0.0	0.866
Learned inventory selector	20260825	1000.0	1000.0	863.0	0.0	0.0	0.717
Learned inventory selector	20260826	1000.0	1000.0	835.6	0.0	0.0	0.717
Learned inventory selector	20260827	1000.0	1000.0	849.9	0.0	0.0	0.715
Finite catalogue oracle	20260825	1000.0	1000.0	841.5	0.0	0.0	0.719
Finite catalogue oracle	20260826	1000.0	1000.0	843.1	0.0	0.0	0.719
Finite catalogue oracle	20260827	1000.0	1000.0	828.1	0.0	0.0	0.719
Shared-null + post-hoc	20260825	183.9	114.3	109.7	109.7	0.3	0.738
Shared-null + post-hoc	20260826	187.5	103.5	98.3	98.3	0.7	0.763
Shared-null + post-hoc	20260827	212.6	125.3	117.5	117.5	0.3	0.761
FACT-matched	20260825	59.2	49.5	48.8	48.8	0.0	0.676
FACT-matched	20260826	81.7	68.7	66.1	66.1	0.0	0.658
FACT-matched	20260827	41.3	37.8	37.4	37.4	0.0	0.668
FACT-generous	20260825	391.0	161.8	161.1	161.1	0.3	0.782
FACT-generous	20260826	747.1	508.8	488.3	488.3	0.0	0.768
FACT-generous	20260827	363.9	219.4	215.5	215.5	0.0	0.771
FORGE Transformer	20260825	801.4	722.7	702.1	702.1	0.0	0.617
FORGE Transformer	20260826	969.1	903.0	843.8	843.8	0.3	0.613
FORGE Transformer	20260827	827.5	734.7	707.7	707.7	0.0	0.634

Table 11: **Reaction decomposition outcomes under the common Ugi verifier.** Coverage is conditioned on molecular validity; replay precision is conditioned on returned decompositions. Abstention and ambiguity retain their prespecified denominators. N/E denotes a conditional metric with no eligible valid product or returned decomposition.

Method	Valid products	Decomp. coverage	Replay precision	Abstention	Ambiguity
RGFN	0.0 ± 0.0	N/E	N/E	N/E	N/E
DeFoG unconditional	1064.3 ± 129.9	81.2 ± 2.4	100.0 ± 0.0	18.8 ± 2.4	0.0 ± 0.0
GenMol/SAFE	1927.7 ± 19.0	0.0 ± 0.0	N/E	100.0 ± 0.0	0.0 ± 0.0
Learned inventory selector	3072.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Finite catalogue oracle	3072.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Shared-null + post-hoc	598.0 ± 47.9	58.8 ± 3.5	100.0 ± 0.0	41.2 ± 3.5	0.0 ± 0.0
FACT-matched	186.7 ± 62.1	86.3 ± 4.4	100.0 ± 0.0	13.7 ± 4.4	0.0 ± 0.0
FACT-generous	1538.0 ± 656.9	56.6 ± 13.7	100.0 ± 0.0	43.4 ± 13.7	0.0 ± 0.0
FORGE Transformer	2660.3 ± 277.1	90.7 ± 2.2	100.0 ± 0.0	9.3 ± 2.2	0.0 ± 0.0

Table 12: **Finite component inventories and source-product coverage.** Inventory size counts unique constitutional components visible to the method at generation time. Product coverage requires that every required role component is present in that inventory. The held-component fold is never used to construct an inventory.

Method	Amine	Aldehyde	Isocyanide	Train coverage	Calibration coverage	Held-comp. coverage
RGFN	185	75	35	100.0	0.0	0.0
DeFoG unconditional	N/P	N/P	N/P	N/P	N/P	N/P
GenMol/SAFE	N/P	N/P	N/P	N/P	N/P	N/P
Finite catalogue oracle	185	75	35	100.0	0.0	0.0
Learned inventory selector	185	75	35	100.0	0.0	0.0
Shared-null + post-hoc	N/P	N/P	N/P	N/P	N/P	N/P
FACT-matched	N/P	N/P	N/P	N/P	N/P	N/P
FACT-generous	N/P	N/P	N/P	N/P	N/P	N/P
FORGE Transformer	N/P	N/P	N/P	N/P	N/P	N/P

E Theoretical properties

This appendix establishes five properties of FORGE. We show that the sparse state space represents every connected labeled graph within the declared vocabulary, size, and cycle-rank support; characterize the reachability ceiling of a finite-component assembler; prove invariance and population consistency for program-masked denoising; quantify the efficiency of post-hoc rejection under a fixed sampling budget; and establish termination and soundness for the bounded recursive route assessor.

These results concern representation, population objectives, and computational verification. They do not imply that finite-data training assigns positive probability to every representable graph, that registered transformations cover arbitrary chemistry, that bounded route search is complete, or that a returned dossier guarantees experimental synthesis success.

E.1 Notation and setup

All graphs in this appendix are finite, connected, undirected, loop-free labeled graphs. Let Σ_V and Σ_E be the finite atom-state and bond-state vocabularies, and define

$$\mathfrak{G}_{N,K} = \{G = (V, E) : |V| \leq N, \beta(G) = |E| - |V| + 1 \leq K, \ell_V(V) \subseteq \Sigma_V, \ell_E(E) \subseteq \Sigma_E\}. \quad (33)$$

Graphs are identified up to label-preserving isomorphism.

A sparse FORGE state with n active nodes is well formed when:

1. $1 \leq n \leq N$, and every active node has a label in Σ_V ;
2. the root has no parent and every node $i > 1$ has one parent $\pi_i < i$;
3. every parent edge has a label in Σ_E ;
4. it contains $k \leq K$ distinct closure edges, each carrying a label in Σ_E , joining two active nodes, and duplicating neither a tree edge nor another closure edge; and
5. closure edges are ordered lexicographically by their endpoint pairs.

Let $\mathfrak{X}_{N,K}$ denote this state space and $\text{Dec} : \mathfrak{X}_{N,K} \rightarrow \mathfrak{G}_{N,K}$ its decoder.

E.2 Sparse graph representation

Lemma E.1 (Tree residual count). *Let $G = (V, E)$ be a connected finite graph and let T be any spanning tree of G . Then*

$$|E \setminus E(T)| = |E| - |V| + 1 = \beta(G). \quad (34)$$

Proof. A spanning tree on $|V|$ vertices contains exactly $|V| - 1$ edges. Because $E(T) \subseteq E$,

$$|E \setminus E(T)| = |E| - |E(T)| = |E| - (|V| - 1) = |E| - |V| + 1. \quad (35)$$

The final expression is the cyclomatic number of a connected graph. $\square$

Theorem E.2 (Bounded representational completeness). *The decoder*

$$\text{Dec} : \mathfrak{X}_{N,K} \longrightarrow \mathfrak{G}_{N,K} \quad (36)$$

is surjective up to label-preserving graph isomorphism. Equivalently:

1. *for every $G \in \mathfrak{G}_{N,K}$, there exists $X \in \mathfrak{X}_{N,K}$ such that $\text{Dec}(X) \cong G$; and*
2. *every $X \in \mathfrak{X}_{N,K}$ decodes to a graph in $\mathfrak{G}_{N,K}$.*

Proof. Let $G = (V, E) \in \mathfrak{G}_{N,K}$, and choose a spanning tree $T \subseteq G$. Choose a root of T and enumerate its vertices $v_1, \dots, v_n$ in any root-first traversal. Every non-root vertex then appears after its parent.

Assign node i the label of v_i . For each $i > 1$, let $\pi_i < i$ be the index of the parent of v_i in T , and assign the corresponding tree-edge label. By Lemma ??, the residual edge set $E \setminus E(T)$ contains $\beta(G) \leq K$ edges. Orient each residual edge by increasing endpoint index, attach its original edge label, and sort the resulting triples lexicographically by endpoint pair. The resulting state is an element of $\mathfrak{X}_{N,K}$.

The decoder reconstructs every edge of T from the parent pointers and every edge of $E \setminus E(T)$ from the closure triples. It also preserves all node and edge labels. The map $i \mapsto v_i$ is therefore a label-preserving isomorphism from $\text{Dec}(X)$ to G .

Conversely, let $X \in \mathfrak{X}_{N,K}$ have n active nodes. Since $\pi_i < i$, repeatedly following parent pointers from any node strictly decreases its index and must terminate at the root. The $n - 1$ parent edges therefore form a connected acyclic graph, hence a spanning tree. If the state contains $k \leq K$ closure edges, the decoded graph has

$$|E| = (n - 1) + k \quad (37)$$

distinct edges. Its cyclomatic number is consequently

$$|E| - |V| + 1 = (n - 1 + k) - n + 1 = k \leq K. \quad (38)$$

The remaining well-formedness conditions preserve the declared node and edge vocabularies and exclude loops or duplicate edges. Thus $\text{Dec}(X) \in \mathfrak{G}_{N,K}$. $\square$

E.3 Finite-component reachability

Theorem E.3 (Finite-component reachability). *Assume that $\mathfrak{T}_P(\mathcal{C})$ is finite and that every forward-outcome set is finite. Then*

$$|\mathcal{G}_{\text{cat}}(P, \mathcal{C})| \leq \sum_{\mathbf{C} \in \mathfrak{T}_P(\mathcal{C})} |\mathcal{F}_P(\mathbf{C})|. \quad (39)$$

If $\mathfrak{T}_P(\mathcal{C}) \neq \emptyset$ and

$$M_P(\mathcal{C}) = \max_{\mathbf{C} \in \mathfrak{T}_P(\mathcal{C})} |\mathcal{F}_P(\mathbf{C})|, \quad (40)$$

then

$$|\mathcal{G}_{\text{cat}}(P, \mathcal{C})| \leq M_P(\mathcal{C}) |\mathfrak{T}_P(\mathcal{C})|. \quad (41)$$

Moreover,

$$G \in \mathcal{G}_{\text{cat}}(P, \mathcal{C}) \iff \mathcal{E}_P^*(G) \cap \mathfrak{T}_P(\mathcal{C}) \neq \emptyset. \quad (42)$$

Every catalogue attempt uses only catalogue components, so its attainable component-novelty fraction is exactly zero.

686

Proof. The reachable set is a finite union of forward-outcome sets. The cardinality of a union is at most the sum of the cardinalities of its members, which proves the first inequality. Bounding each summand by $M_P(\mathcal{C})$ proves the second.

By definition, G belongs to the union precisely when at least one $\mathbf{C} \in \mathfrak{T}_P(\mathcal{C})$ satisfies $G \in \mathcal{F}_P(\mathbf{C})$. This is equivalent to membership in $\mathcal{E}_P^*(G) \cap \mathfrak{T}_P(\mathcal{C})$. Finally, every component in an input from $\mathfrak{T}_P(\mathcal{C})$ belongs to its role-specific catalogue by construction, so component novelty is impossible. $\square$

E.4 Program-masked discrete flow

For a program P , let $\mathcal{I}(P)$ index the categorical coordinates of its sampling layout. Define

$$\mathcal{S}(P) = \left\{ (q, j) \in \mathcal{I}(P) : m_j^{(q)}(P) = 0 \right\} \quad (43)$$

and let $x_{P,j}^{(q)}$ be the prescribed value of each fixed coordinate. The program-compatible face of the raw categorical state space is

$$\Omega_P = \left\{ x : x_j^{(q)} = x_{P,j}^{(q)} \text{ for every } (q, j) \in \mathcal{S}(P) \right\}. \quad (44)$$

698 Define the projection Π_P coordinatewise by

$$[\Pi_P(x)]_j^{(q)} = \begin{cases} x_{P,j}^{(q)}, & (q, j) \in \mathcal{S}(P), \\ x_j^{(q)}, & (q, j) \notin \mathcal{S}(P). \end{cases} \quad (45)$$

699 Training corruption has the form $X_t = \Pi_P(\tilde{X}_t)$. Numerical sampling begins from $X_0 =$
700 $\Pi_P(\tilde{X}_0)$ and applies

$$X_{k+1} = \Pi_P(\Phi_k^\theta(X_k, \xi_k)), \quad k = 0, \dots, L-1, \quad (46)$$

701 where Φ_k^θ is the unconstrained stochastic update and ξ_k contains its sampling randomness.

Proposition E.4 (Program-fixed invariance). *For every program P ,*

$$\Pr[X_t \in \Omega_P] = 1 \quad (47)$$

at every corruption time t , and

$$\Pr[X_k \in \Omega_P \text{ for every } k = 0, \dots, L] = 1 \quad (48)$$

during numerical sampling.

702

703 *Proof.* By definition, $\Pi_P(x) \in \Omega_P$ for every raw state x , and $\Pi_P(x) = x$ for every $x \in \Omega_P$.
704 Since corruption returns $\Pi_P(\tilde{X}_t)$, every corrupted state lies in Ω_P .

705 The sampling claim follows by induction. The projected initial state belongs to Ω_P . If
706 $X_k \in \Omega_P$, the unconstrained update may change any coordinate, but applying Π_P restores
707 every coordinate in $\mathcal{S}(P)$. Hence $X_{k+1} \in \Omega_P$. The conclusion holds for all sampling steps. $\square$

708 Define the joint population law

$$\nu(dG, dP, dt, dX_t) = \mu_{\text{bal}}(dG, dP) \mathcal{U}(dt) p_t(dX_t \mid X_G, P). \quad (49)$$

709 For a variable coordinate, let

$$\rho_{qj}(a \mid X_t, t, P) = \Pr_\nu[X_{G,j}^{(q)} = a \mid X_t, t, P]. \quad (50)$$

Theorem E.5 (Population optimality of masked denoising). *Assume an unrestricted conditional model class and $\lambda_q > 0$. For every coordinate satisfying $m_j^{(q)}(P) = 1$, every population minimizer of $\mathcal{L}_{\text{FORGE}}$ satisfies*

$$p_\theta^*(X_{G,j}^{(q)} = a \mid X_t, t, P) = \rho_{qj}(a \mid X_t, t, P) \quad (51)$$

for every categorical state a , ν -almost surely. Coordinates with $m_j^{(q)}(P) = 0$ contribute no loss and are instead governed by Proposition ??.

710

711 *Proof.* Fix a family q , a variable coordinate j , and a context $C = (X_t, t, P)$ for which the
712 regular conditional distribution is defined. Let $p(a) = p_\theta(X_{G,j}^{(q)} = a \mid C)$ and $\rho(a) = \rho_{qj}(a \mid C)$.
713 The conditional expected loss is

$$-\sum_a \rho(a) \log p(a) = -\sum_a \rho(a) \log \rho(a) + \sum_a \rho(a) \log \frac{\rho(a)}{p(a)} \quad (52)$$

$$= H(\rho) + D_{\text{KL}}(\rho \parallel p). \quad (53)$$

714 The entropy term is independent of p . By non-negativity of Kullback–Leibler divergence,
715 the expression is minimized exactly when $p = \rho$.

716 For fixed P and q , the factor $\lambda_q / \sum_j m_j^{(q)}(P)$ is positive and independent of the predicted
717 categorical distribution. It therefore does not change the pointwise minimizer. Integrating
718 over contexts and repeating the argument for every variable coordinate proves the result.
719 Coordinates with zero masks do not appear in the objective. $\square$

720 **E.5 Matched-budget post-hoc filtering**

721 Fix a reaction program P , and define

$$A_P(G) = \mathbb{I}[G \in \mathcal{G}_{L1}(P)]. \quad (54)$$

722 For a proposal distribution Q on the bounded graph space, let

$$Z_Q(P) = \Pr_{G \sim Q}[A_P(G) = 1] = \mathbb{E}_{G \sim Q}[A_P(G)] \quad (55)$$

723 denote the exact-L1 acceptance probability.

Proposition E.6 (Exact law of matched-budget post-hoc filtering). *Suppose $Z_Q(P) > 0$. Rejection filtering produces the conditional distribution*

$$Q_{\text{post}}(g \mid P) = \Pr_{G \sim Q}[G = g \mid A_P(G) = 1] = \frac{Q(g)A_P(g)}{Z_Q(P)}. \quad (56)$$

For B independent attempts $G_1, \dots, G_B \sim Q$, define

$$N_B = \sum_{b=1}^B A_P(G_b). \quad (57)$$

Then

$$N_B \sim \text{Binomial}(B, Z_Q(P)), \quad (58)$$

and consequently

$$\mathbb{E}[N_B] = BZ_Q(P), \quad (59)$$

$$\text{Var}(N_B) = BZ_Q(P)(1 - Z_Q(P)), \quad (60)$$

$$\Pr[N_B = 0] = (1 - Z_Q(P))^B. \quad (61)$$

If T_K is the number of independent attempts required to obtain K accepted products, then

$$\mathbb{E}[T_K] = \frac{K}{Z_Q(P)}. \quad (62)$$

Finally, for a program-conditioned proposal Q_P and an unconditioned proposal Q_0 ,

$$\mathbb{E}[N_B^P] - \mathbb{E}[N_B^0] = B(Z_{Q_P}(P) - Z_{Q_0}(P)). \quad (63)$$

724

725 *Proof.* For any graph g , Bayes' rule gives

$$\Pr[G = g \mid A_P(G) = 1] = \frac{\Pr[A_P(G) = 1 \mid G = g] \Pr[G = g]}{\Pr[A_P(G) = 1]} \quad (64)$$

$$= \frac{A_P(g)Q(g)}{Z_Q(P)}. \quad (65)$$

726 Since the proposals are independent, the indicators $A_P(G_1), \dots, A_P(G_B)$ are independent
 727 Bernoulli random variables with common success probability $Z_Q(P)$. Their sum is therefore
 728 binomial, which gives its expectation, variance, and zero-acceptance probability.

729 The waiting time T_K has the negative-binomial distribution with success probability $Z_Q(P)$,
 730 whose expectation is $K/Z_Q(P)$. Applying the binomial expectation separately to Q_P and
 731 Q_0 and subtracting proves the final identity. $\square$

732 The proposition compares sampling efficiency only. It does not imply that conditioning
 733 improves $Z_Q(P)$; the matched production experiment tests that empirical claim.

734 E.6 Recursive route construction

Lemma E.7 (Termination of bounded route assessment). *Fix an assessment contract $\mathcal{B}$ with finite maximum route depth $D_{\mathcal{B}}$. Assume that each evidence query terminates and returns either a non-expansion outcome or an expansion with finitely many reactants. Then $\mathcal{A}_{\mathcal{B}}(G, P)$ terminates with either a finite route tree or a typed failure state.*

Proof. We use induction on the remaining depth $d = D_{\mathcal{B}} - k$ at a node of depth k . When $d = 0$, the assessor performs the terminating evidence query and either accepts a terminal node, returns its typed failure, or rejects an expansion with a depth-exhaustion state. Suppose the claim holds whenever fewer than $d > 0$ levels remain. At the current node, the evidence query either returns a non-expansion outcome immediately or a finite reactant tuple. In the latter case, each child is assessed at depth $k + 1$ and therefore has $d - 1$ levels remaining. Every child assessment terminates with a finite result by the induction hypothesis, and a finite tuple of finite child trees forms a finite parent tree. Finite expansion, proposal, verifier, and time budgets can only stop this recursion earlier. Rejecting an ancestor identity additionally prevents a circular molecular dependency from being accepted, but finite depth alone establishes termination. $\square$

Theorem E.8 (Soundness of returned synthesis dossiers). *Fix $\mathcal{B}$ as in Lemma ?? . If $\mathcal{A}_{\mathcal{B}}(G, P) = T$, then T is finite and $\text{Complete}(G, T, P) = 1$. In particular, G passes exact L1 verification under P , every L2 reaction in T reconstructs its target exactly under its registered forward transform, and every leaf has a current accepted record identifying the same constitutional structure as supplier-listed and available. A candidate tree with $\text{Complete}(G, T, P) = 0$ cannot be returned as a synthesis dossier. The outcome $\mathcal{A}_{\mathcal{B}}(G, P) = \perp$ states only that no complete dossier was returned under $\mathcal{B}$. It does not prove that no route exists under a larger search budget, a different admitted reaction set, or a later supplier snapshot.*

Proof. Finiteness follows from Lemma ?? . It remains to establish the three factors in the completeness predicate. The assessor begins recursive route construction only after exact L1 decomposition and forward replay have succeeded, so a returned dossier satisfies $\text{L1}(G, P) = 1$.

We next use structural induction on the finite route tree. At a leaf, the assessor returns a complete node only when the evidence policy admits an exact terminal-material record whose availability state is current and closed. Therefore $\text{Available}(\ell) = 1$ for every leaf ℓ of a returned tree. At an internal node, the assessor admits an expansion only when the registered reaction has exact-source, exact-substrate evidence and its deterministic forward transform uniquely reconstructs the target product. Thus $\text{Verify}(r) = 1$ for the associated reaction node r . The internal node is complete only when every child is complete; the induction hypothesis therefore applies to all descendant reactions and leaves.

All factors in $\text{Complete}(G, T, P)$ equal one, so their product equals one. Conversely, if any factor for a candidate tree equals zero, the fail-closed return rule does not admit that tree as a dossier and instead records the corresponding typed failure. Because the search and evidence contract is bounded, this rejection establishes no completeness claim about routes outside $\mathcal{B}$. $\square$

766 F Generated-structure atlas

767 Table ?? reports 12 deterministic display examples from the same frozen seed-0 Ugi population
 768 used in Figure ??. The display begins with the lowest constitutional SHA-256 rank and then
 769 applies ECFP4 MaxMin selection among the 23 eligible, constitutionally unique products.
 770 This promotes visual coverage without using activity, route closure, procurement evidence
 771 or human preference. Each row includes the canonical constitutional SMILES so that the
 772 displayed identity is explicit; the exact strings are also stored in the hash-pinned atlas receipt.

Table 13: **FORGE generated-structure atlas.** The 12 rows form one multipage table, indexed by display order. Each constitutional graph is followed by its canonical SMILES. The decorated three-dimensional column shows one deterministic post-hoc RDKit ETKDGv3 structure optimized with up to 400 MMFF94s iterations. Purple denotes the amine-derived head, teal the aldehyde-derived body and tail, orange the isocyanide-derived tail and gray the assembly-introduced oxygen; these assignments are recovered through exact atom-mapped forward replay. FORGE does not generate three-dimensional coordinates, and these views are not predicted bioactive conformations. Exact L1 replay and descriptor-manifold membership are structural diagnostics, not evidence of activity, route closure or synthesis success. These are deterministic display examples, not experimental candidates.

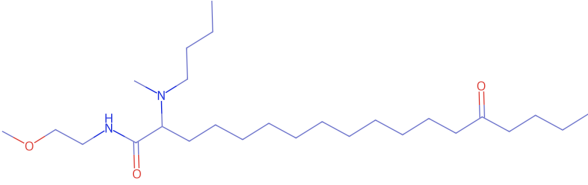
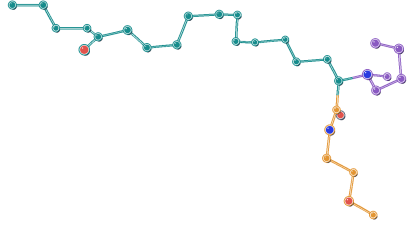
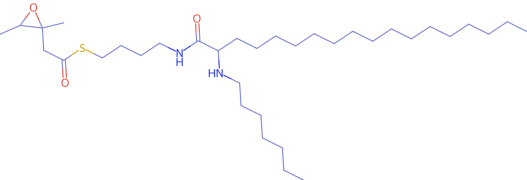
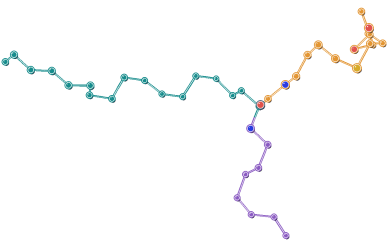
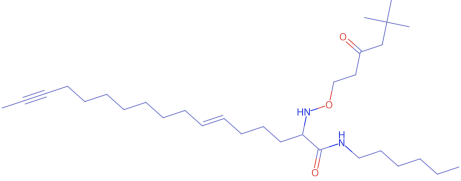
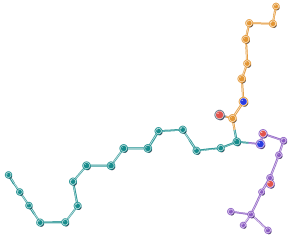
	CONSTITUTIONAL GRAPH AND CANONICAL SMILES	DECORATED 3D VIEW
1	 <chem>CCCCC(=O)CCCCCCCCCCCC(C(=O)NCCCOC)N(C)CCCC</chem>	
2	 <chem>CCCCCCCCCCCCCCCC(NCCCCCCC)C(=O)NCCCSC(=O)CC1(C)OC1C</chem>	
3	 <chem>CC#CCCCCCCC=CCCCC(NCCCC(=O)CC(C)(C)C)C(=O)NCCCCC</chem>	

Table ??, continued

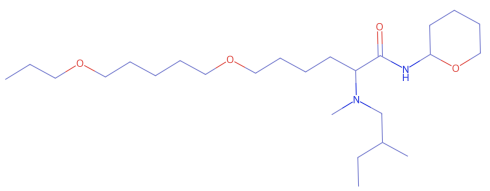
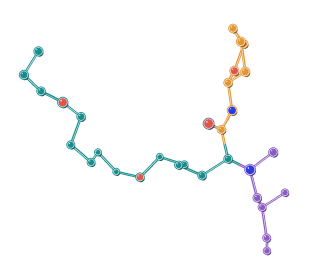
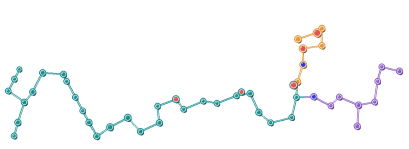
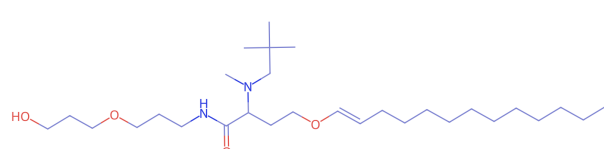
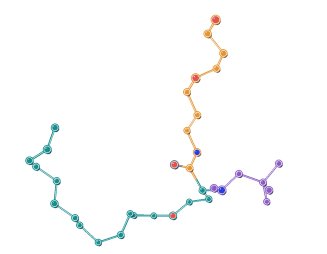
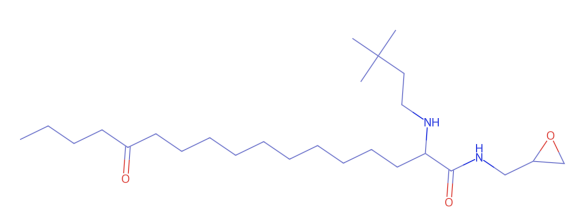
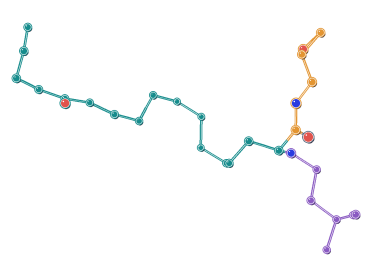
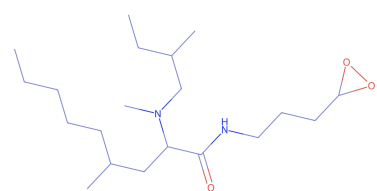
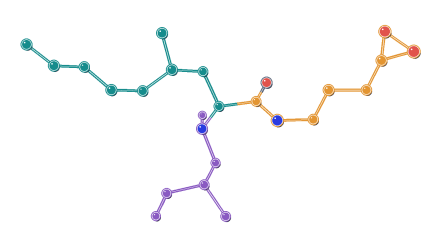
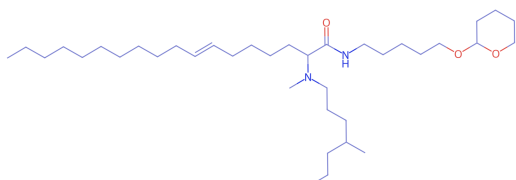
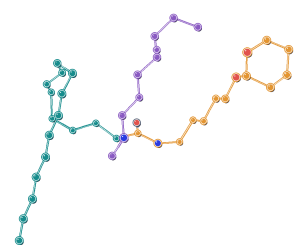
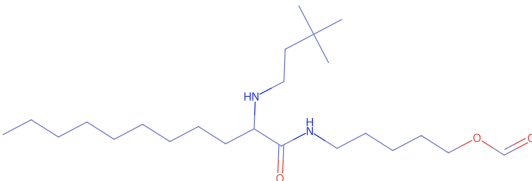
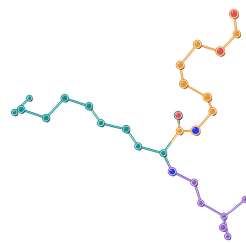
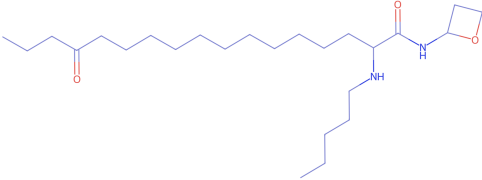
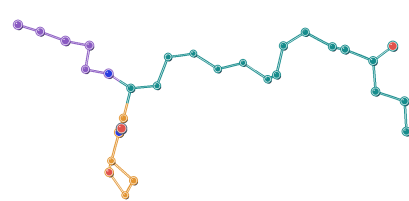
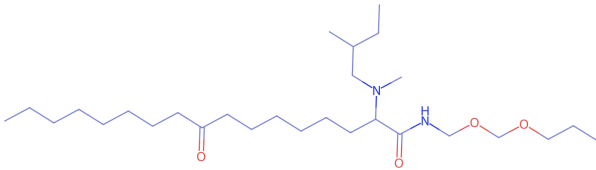
	CONSTITUTIONAL GRAPH AND CANONICAL SMILES	DECORATED 3D VIEW
4	 <chem>CCCCCCCCCCCCC(C=O)N1CCCC1)N(C)CC(C)CC</chem>	
5	 <chem>C#CCC(CC)CCCC=CC#CCCCCCCCCCCCC(=O)CCCC(NCCC(C)CCCC)C(=O)NCCCOC</chem>	
6	 <chem>CCCCCCCCCCCCC=CCCC(C(=O)NCCCCCCCC)N(C)CC(C)(C)C</chem>	
7	 <chem>CCCCC(=O)CCCCCCCCCCCC(NCCC(C)(C)C)C(=O)NCC1CO1</chem>	
8	 <chem>CCCCC(C)CC(C(=O)NCCCC1OO1)N(C)CC(C)CC</chem>	

Table ??, continued

	CONSTITUTIONAL GRAPH AND CANONICAL SMILES	DECORATED 3D VIEW
9	 <chem>CCCCCCCCCCC=CCCCC(C(=O)NCCCCCOC1CCCCO1)N(C)CCCC(C)CCC</chem>	
10	 <chem>CCCCCCCCC(NCCC(C)(C)C)C(=O)NCCCCCOC=O</chem>	
11	 <chem>CCCCCNC(CCCCCCCCCC(=O)CCC)C(=O)NC1CCO1</chem>	
12	 <chem>CCCCCCCCC(=O)CCCCC(C(=O)NCCOC(C)CC)N(C)CC(C)CC</chem>	